Open Source Summit North America 2024

Sharing Reset GPIOs in the Linux Kernel

Krzysztof Kozlowski, Linaro
krzysztof.kozlowski@linaro.org
@krzk@social.kernel.org

Introduction

- Krzysztof Kozlowski
- I work for Linaro in Qualcomm Landing Team

Introduction

- Krzysztof Kozlowski
- I work for Linaro in Qualcomm Landing Team
- I am the co-maintainer (with Rob and Conor) of Devicetree bindings in the Linux kernel
  - I also maintain other Linux kernel pieces
    - Memory controller drivers, NFC, 1-Wire subsystem
    - Samsung Exynos SoC ARM/ARM64 architecture
      - Do you know it covers also Google Tensor SoC in Google Pixel 6 phone?
  - I am also one of most active contributors to the Linux kernel

What This Talk is Going to Be About?

- Problem - sharing a reset GPIO

What This Talk is Going to Be About?

- Problem - sharing a reset GPIO
- Non-exclusive GPIO and the Regulator way

What This Talk is Going to Be About?

- Problem - sharing a reset GPIO
- Non-exclusive GPIO and the Regulator way
- Reset controller framework way

What This Talk is Going to Be About?

- Problem - sharing a reset GPIO
- Non-exclusive GPIO and the Regulator way
- Reset controller framework way
- GPIO Aggregator and delay idea

What This Talk is Going to Be About?

- Problem - sharing a reset GPIO
- Non-exclusive GPIO and the Regulator way
- Reset controller framework way
- GPIO Aggregator and delay idea
- Maybe we need GPIO-enable-counting?

What This Talk is Going to Be About?

- Problem - sharing a reset GPIO
- Non-exclusive GPIO and the Regulator way
- Reset controller framework way
- GPIO Aggregator and delay idea
- Maybe we need GPIO-enable-counting?
- Talk focuses on the Devicetree use cases (was submitted to EOSS)
  - Few solutions are Devicetree specific
  - There will be snippets of Devicetree Sources code
  - I don’t know how ACPI is handling this
    - Probably does not handle

Problem: Sharing a reset GPIO

- GPIO - General-Purpose Input/Output, usually coming from System on Chip (SoC) or microcontroller

Problem: Sharing a reset GPIO

- GPIO - General-Purpose Input/Output, usually coming from System on Chip (SoC) or microcontroller
  - Reset GPIO, also sometimes shutdown or powerdown GPIO, is responsible for cutting power to a device

Problem: Sharing a reset GPIO

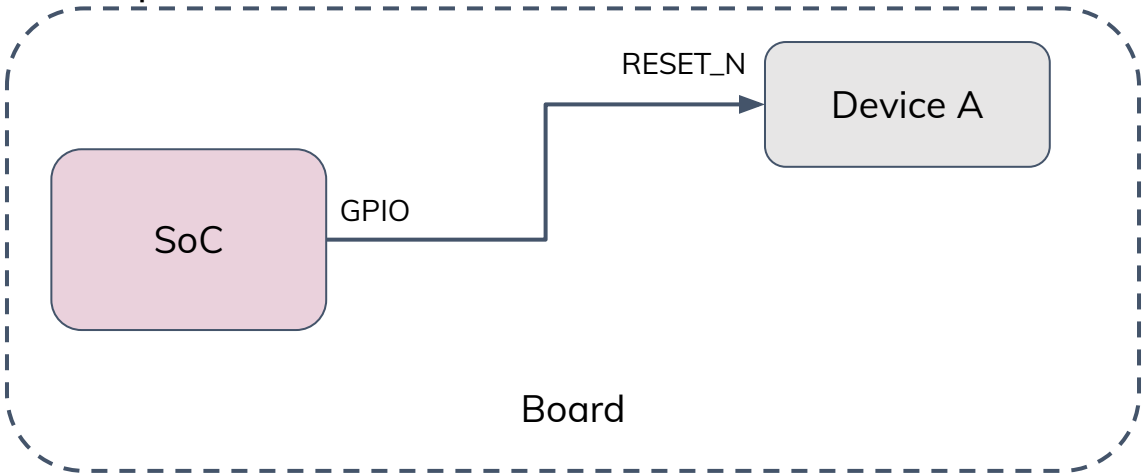
- GPIO - General-Purpose Input/Output, usually coming from System on Chip (SoC) or microcontroller
  - Reset GPIO, also sometimes shutdown or powerdown GPIO, is responsible for cutting power to a device
  - Practically, reset and shutdown are different cases, but problem is the same here

Problem: Sharing a reset GPIO

- GPIO - General-Purpose Input/Output, usually coming from System on Chip (SoC) or microcontroller
  - Reset GPIO, also sometimes shutdown or powerdown GPIO, is responsible for cutting power to a device
  - Practically, reset and shutdown are different cases, but problem is the same here
- From time to time hardware engineers come with design where one such GPIO goes to two independent devices on an embedded board

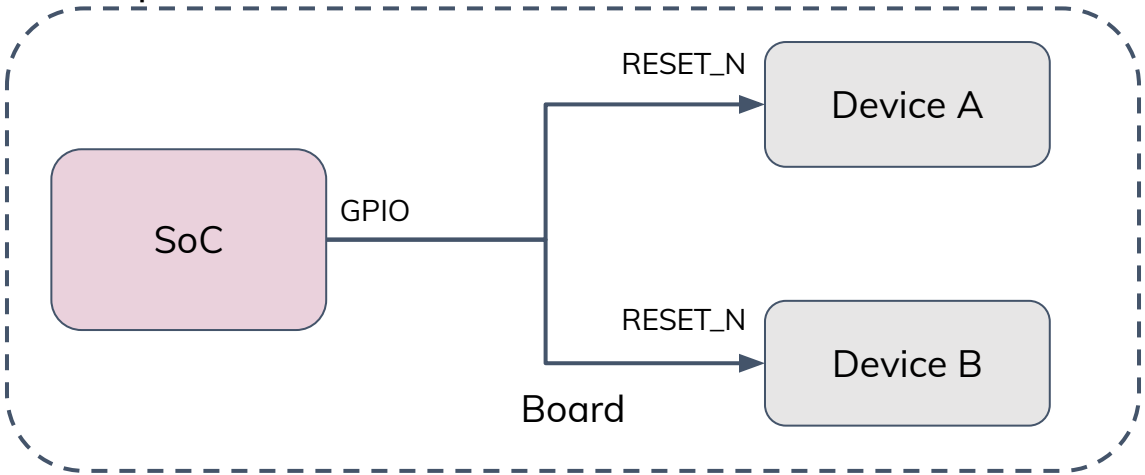
Problem: Sharing a reset GPIO

- GPIO - General-Purpose Input/Output, usually coming from System on Chip (SoC) or microcontroller
  - Reset GPIO, also sometimes shutdown or powerdown GPIO, is responsible for cutting power to a device
  - Practically, reset and shutdown are different cases, but problem is the same here
- From time to time hardware engineers come with design where one such GPIO goes to two independent devices on an embedded board

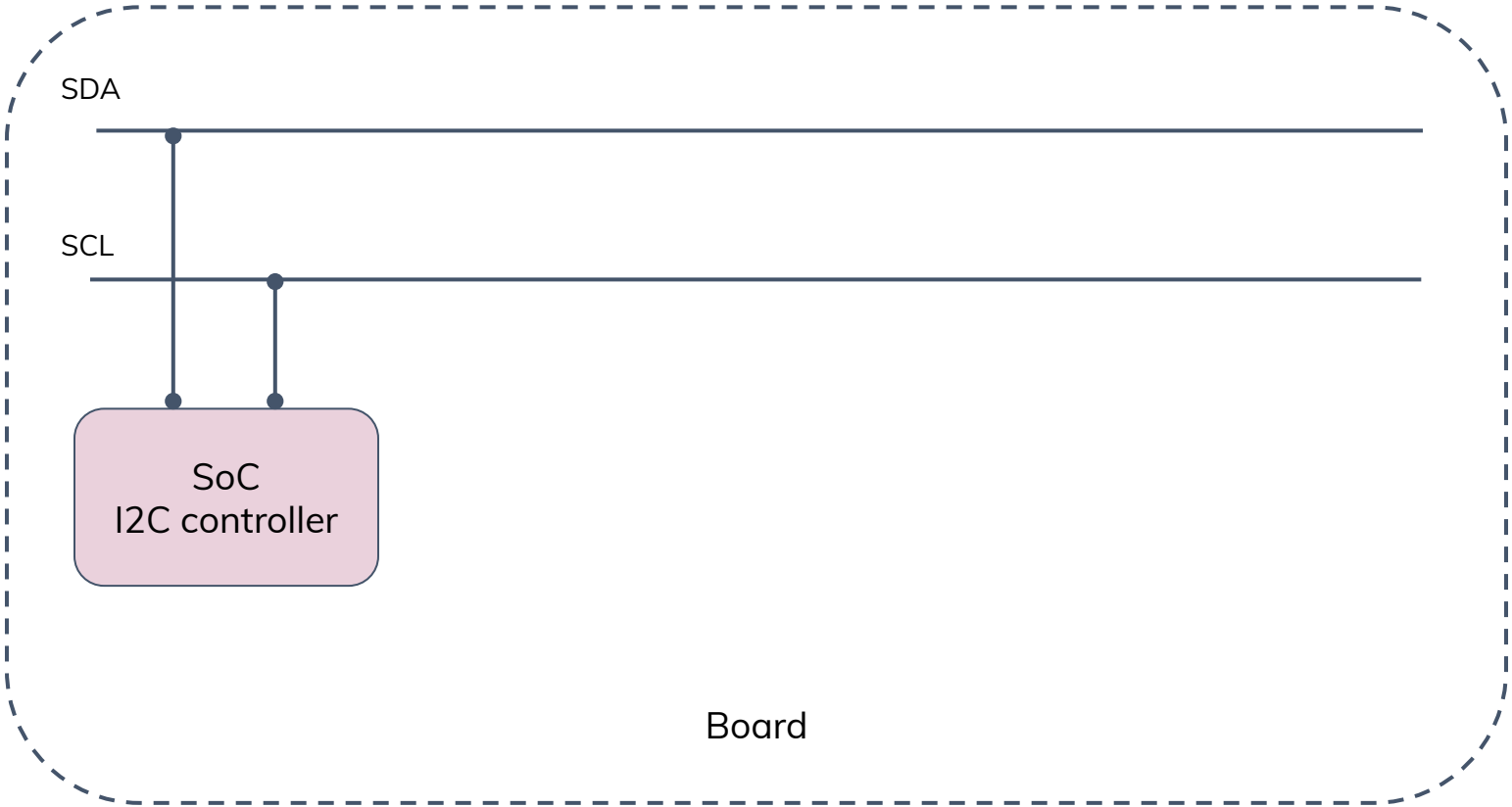


Problem: Sharing a reset GPIO

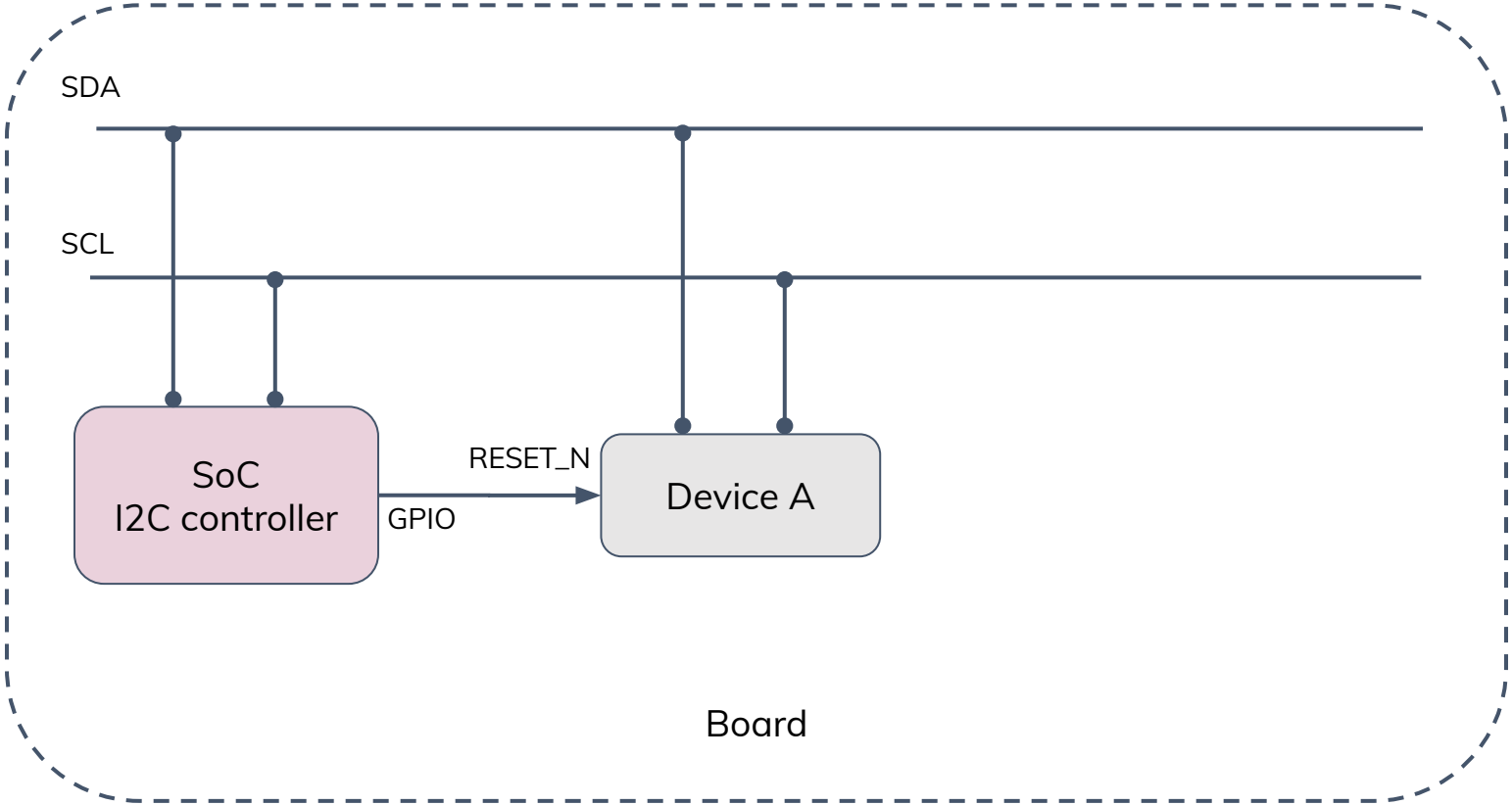
- GPIO - General-Purpose Input/Output, usually coming from System on Chip (SoC) or microcontroller
  - Reset GPIO, also sometimes shutdown or powerdown GPIO, is responsible for cutting power to a device
  - Practically, reset and shutdown are different cases, but problem is the same here
- From time to time hardware engineers come with design where one such GPIO goes to two independent devices on an embedded board



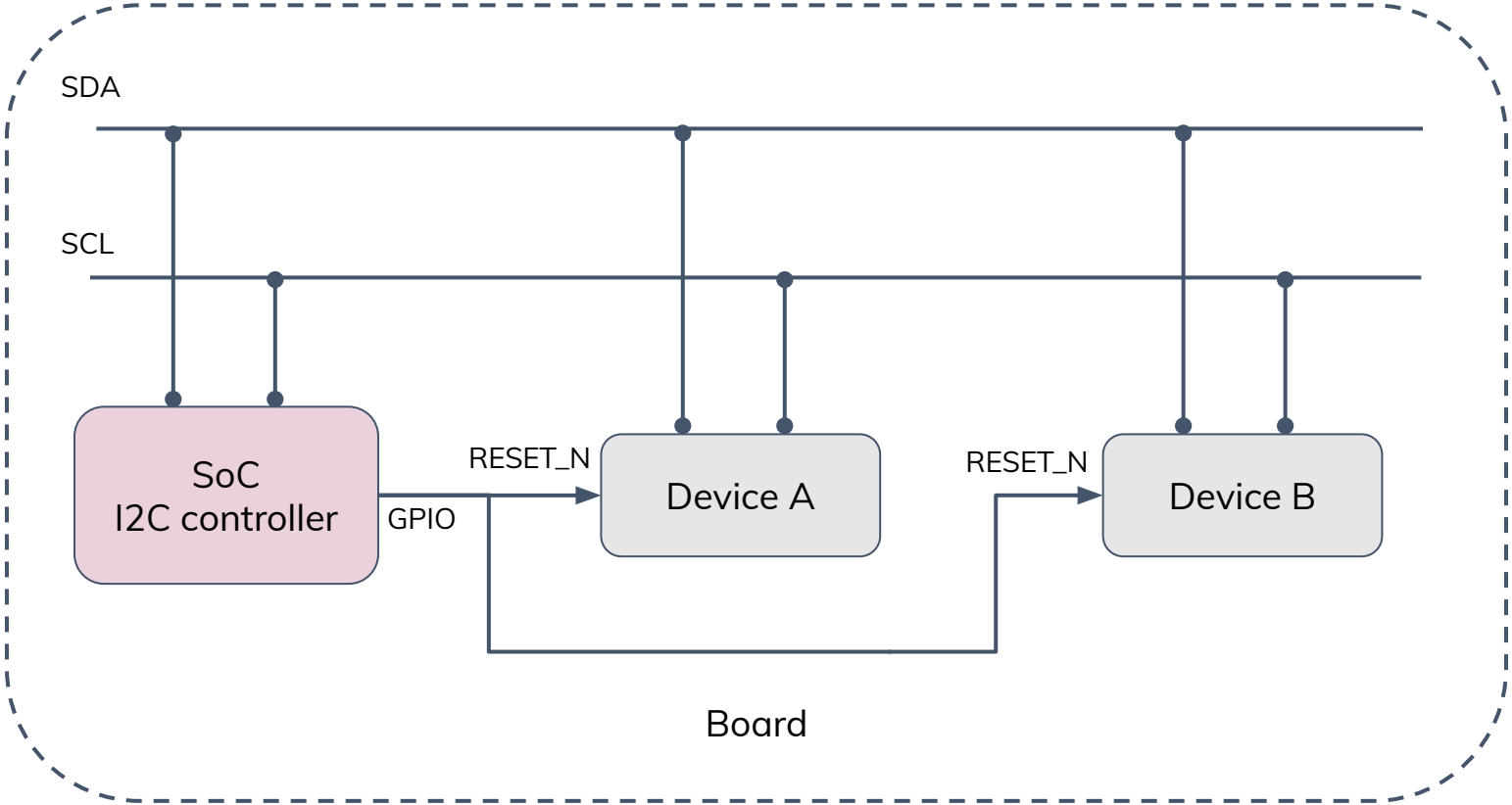
Example: I2C



Example: I2C



Example: I2C


SDA
SCL
SoC
I2C controller
GPIO
RESET_N
Device A
RESET_N
Device B
Board

One Reset to Rule Them All

- Device A and Device B could be the same type of Device...
  - Same Linux driver, but still two Linux devices as in Linux Driver Model (“struct device”)

One Reset to Rule Them All

- Device A and Device B could be the same type of Device...
  - Same Linux driver, but still two Linux devices as in Linux Driver Model (“struct device”)
- Or could be two entirely different devices

One Reset to Rule Them All

- Device A and Device B could be the same type of Device...
  - Same Linux driver, but still two Linux devices as in Linux Driver Model (“struct device”)
- Or could be two entirely different devices
- Does not have to be two! Could be three devices with the same reset signal

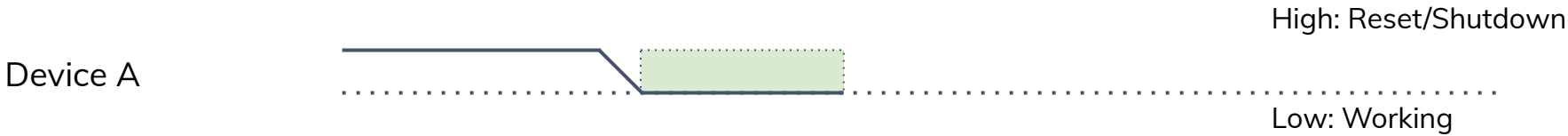
One Reset to Rule Them All

- Device A and Device B could be the same type of Device...
  - Same Linux driver, but still two Linux devices as in Linux Driver Model (“struct device”)
- Or could be two entirely different devices
- Does not have to be two! Could be three devices with the same reset signal
- When reset is asserted, all devices will go to reset

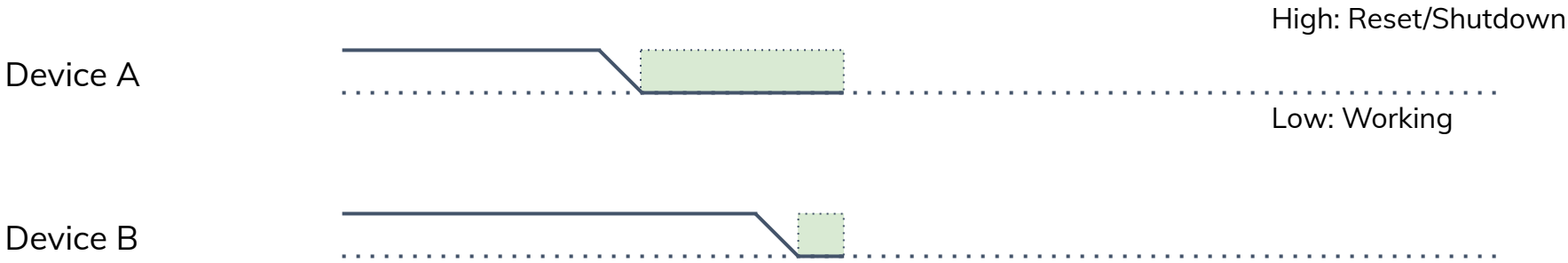
One Reset to Rule Them All

- Device A and Device B could be the same type of Device...
  - Same Linux driver, but still two Linux devices as in Linux Driver Model (“struct device”)
- Or could be two entirely different devices
- Does not have to be two! Could be three devices with the same reset signal
- When reset is asserted, all devices will go to reset
- Same the other way: de-assertion will turn on all devices

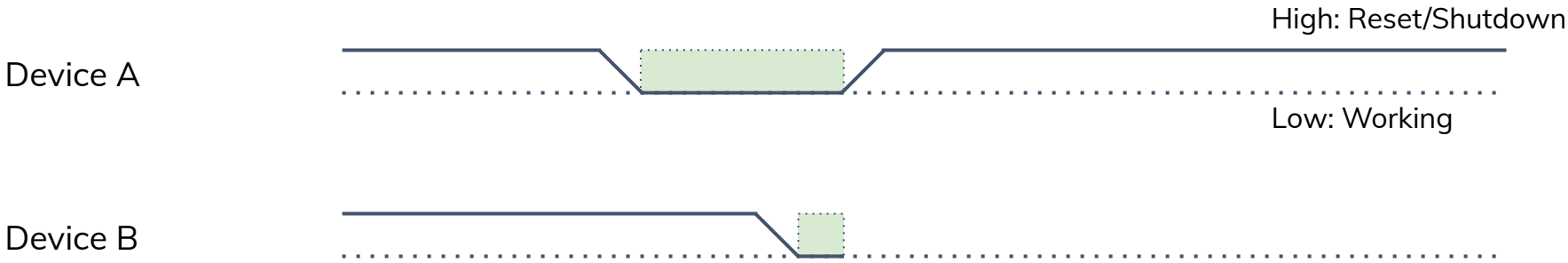
Shared Reset or Shutdown Line, Active High



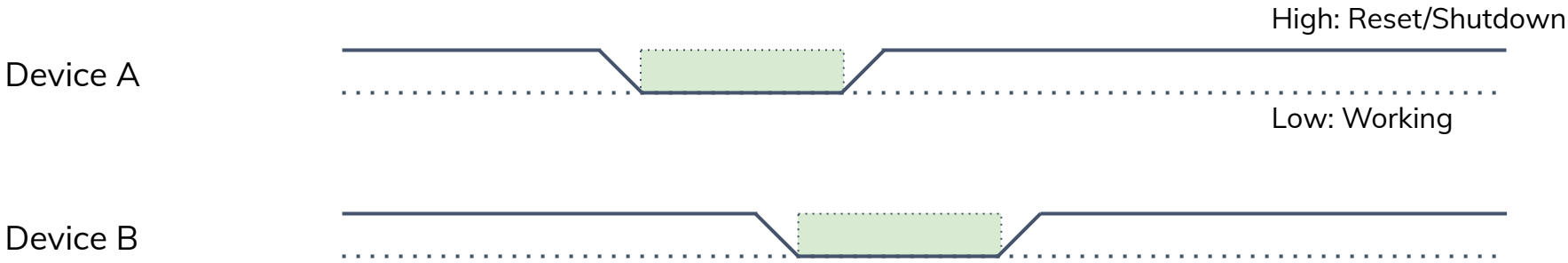
Shared Reset or Shutdown Line, Active High



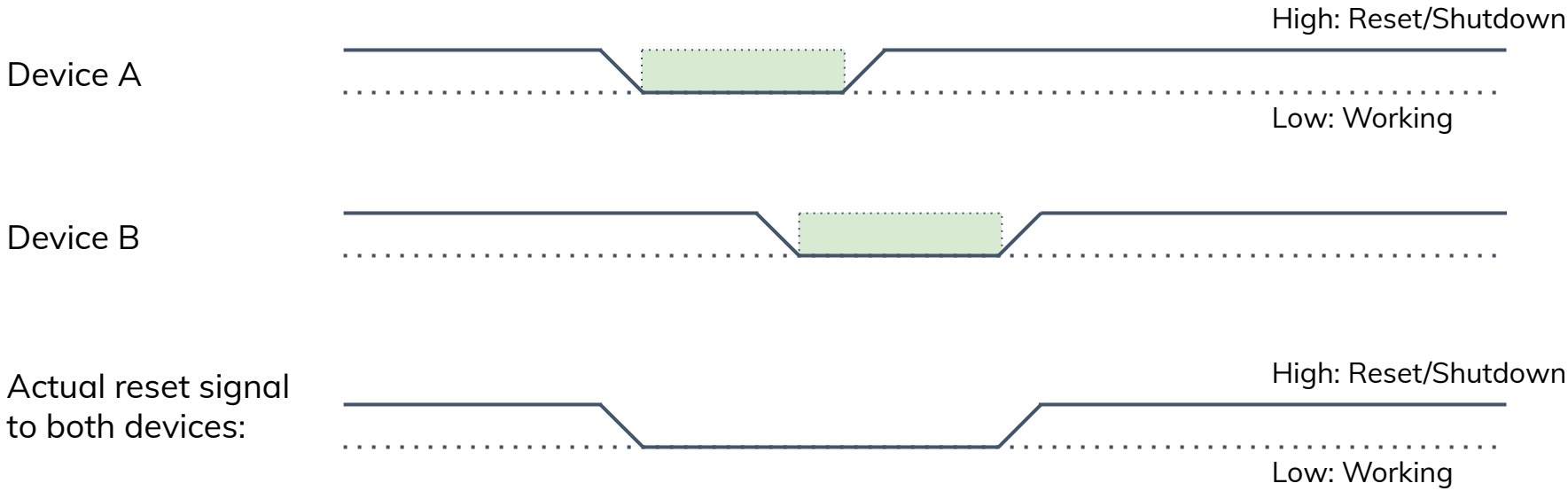
Shared Reset or Shutdown Line, Active High



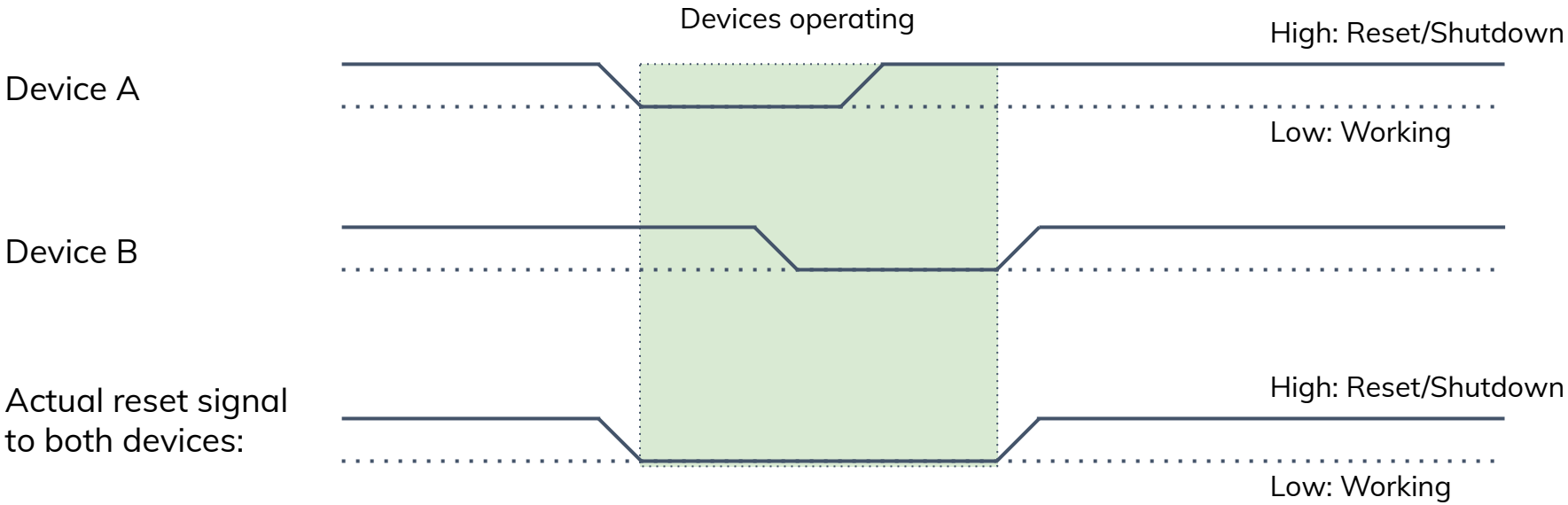
Shared Reset or Shutdown Line, Active High



Shared Reset or Shutdown Line, Active High



Shared Reset or Shutdown Line, Active High

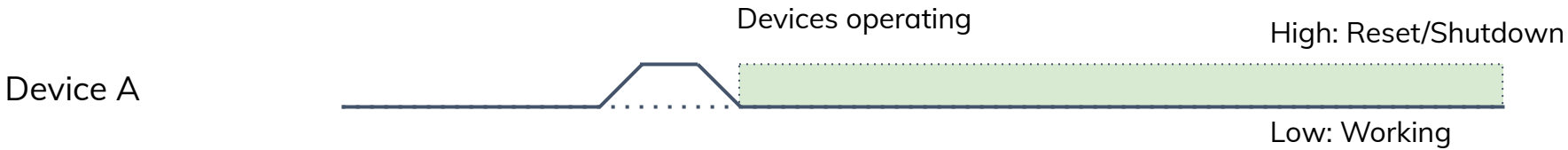


- Reset or shutdown lines often are active low, but for simplicity let's skip this detail

Shared Reset Pulse, Active High

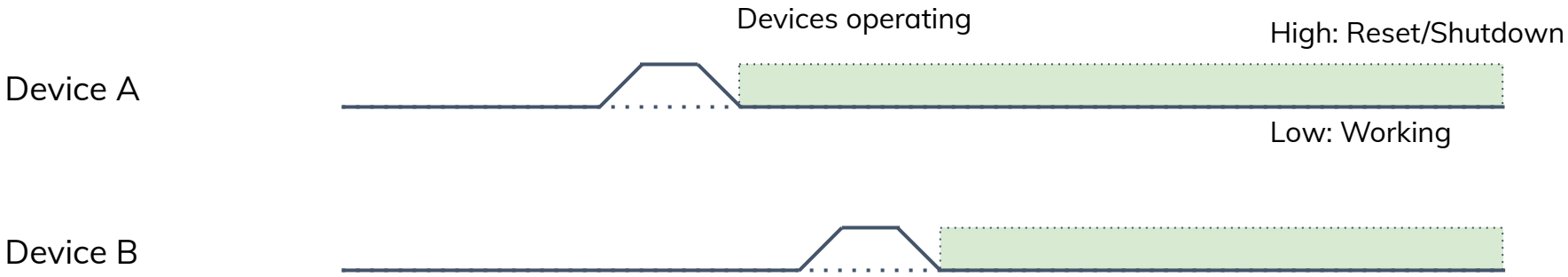
- Reset or shutdown lines often are active low, but for simplicity let's skip this detail

Shared Reset Pulse, Active High



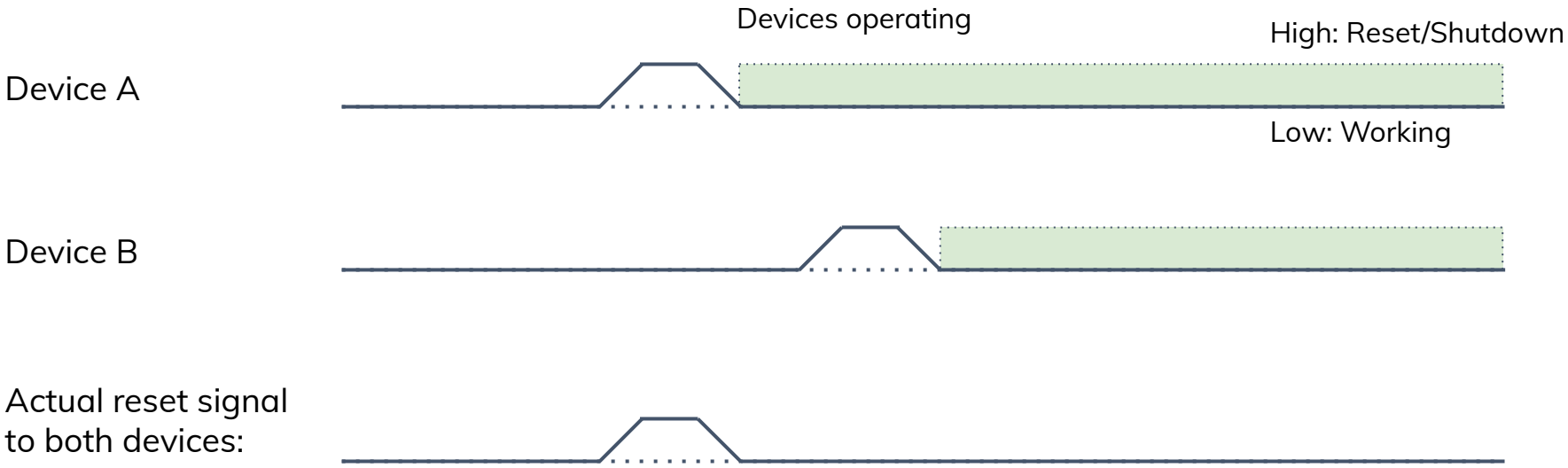
- Reset or shutdown lines often are active low, but for simplicity let's skip this detail

Shared Reset Pulse, Active High



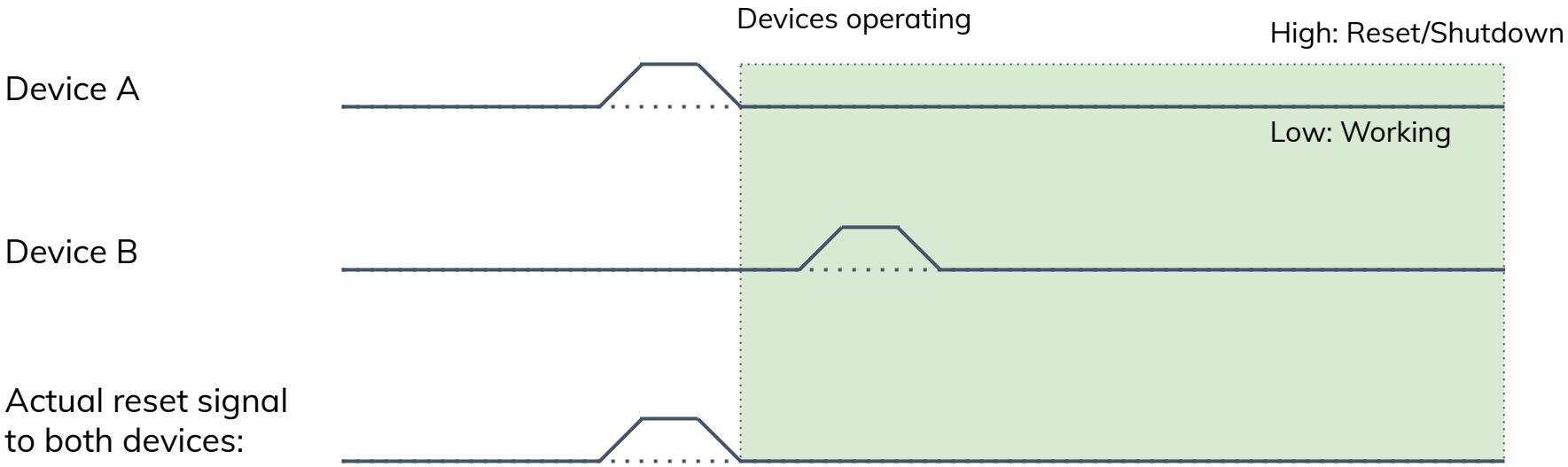
- Reset or shutdown lines often are active low, but for simplicity let's skip this detail

Shared Reset Pulse, Active High



- Reset or shutdown lines often are active low, but for simplicity let's skip this detail

Shared Reset Pulse, Active High



- Reset or shutdown lines often are active low, but for simplicity let's skip this detail

Why Sharing Is Not Caring?

- Some devices might have specific timings or requirements
  - Configure clock rate within X ms after reset deassertion

Why Sharing Is Not Caring?

- Some devices might have specific timings or requirements
  - Configure clock rate within X ms after reset deassertion
  - OS should not deassert your reset, just because other device is doing so
    - At least not without some sort of coordination

Why Sharing Is Not Caring?

- Some devices might have specific timings or requirements
  - Configure clock rate within X ms after reset deassertion
  - OS should not deassert your reset, just because other device is doing so
    - At least not without some sort of coordination
- Typical Linux driver does not handle it well

Why Sharing Is Not Caring?

- Some devices might have specific timings or requirements
  - Configure clock rate within X ms after reset deassertion
  - OS should not deassert your reset, just because other device is doing so
    - At least not without some sort of coordination
- Typical Linux driver does not handle it well
  - Most of drivers are written with assumption they are sole owners of GPIOs

Why Sharing Is Not Caring?

- Some devices might have specific timings or requirements
  - Configure clock rate within X ms after reset deassertion
  - OS should not deassert your reset, just because other device is doing so
    - At least not without some sort of coordination
- Typical Linux driver does not handle it well
  - Most of drivers are written with assumption they are sole owners of GPIOs
  - Particular device driver can be used in cases with and without shared reset GPIOs

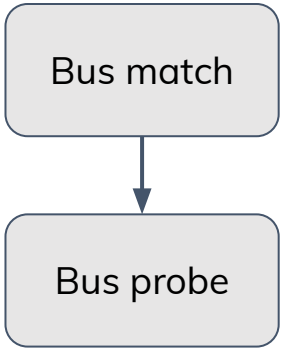
Why Sharing Is Not Caring?

- Some devices might have specific timings or requirements
  - Configure clock rate within X ms after reset deassertion
  - OS should not deassert your reset, just because other device is doing so
    - At least not without some sort of coordination
- Typical Linux driver does not handle it well
  - Most of drivers are written with assumption they are sole owners of GPIOs
  - Particular device driver can be used in cases with and without shared reset GPIOs
- Linux GPIO framework does not even allow multiple drivers to request the same GPIO
  - More on that later

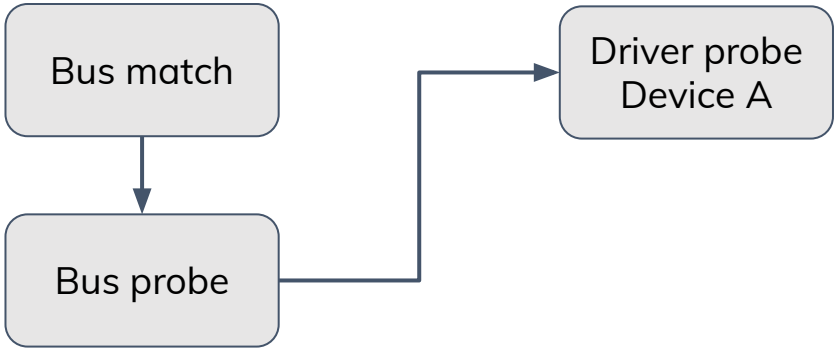
(Simplified) Linux Device Driver Probing

Bus match

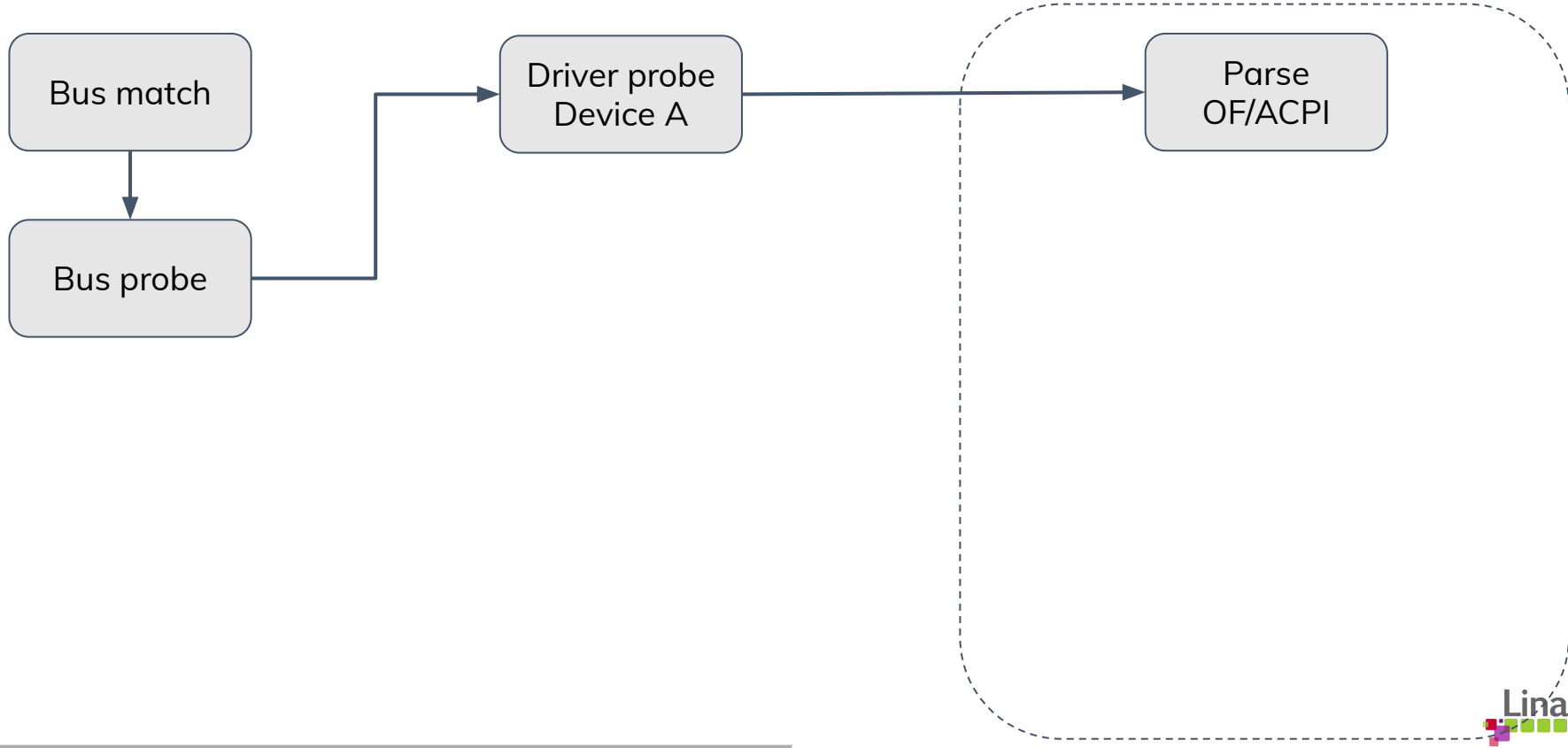
(Simplified) Linux Device Driver Probing



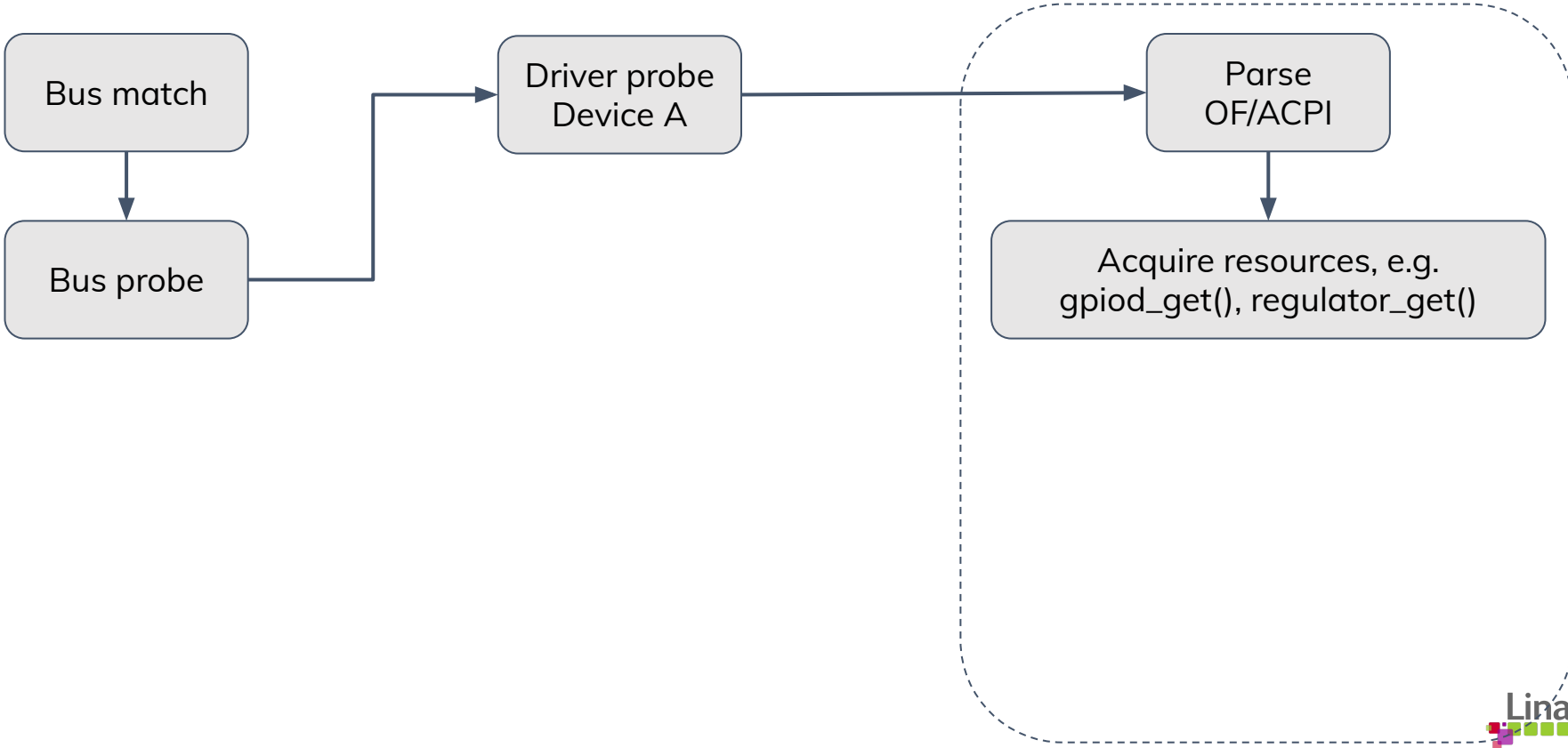
(Simplified) Linux Device Driver Probing



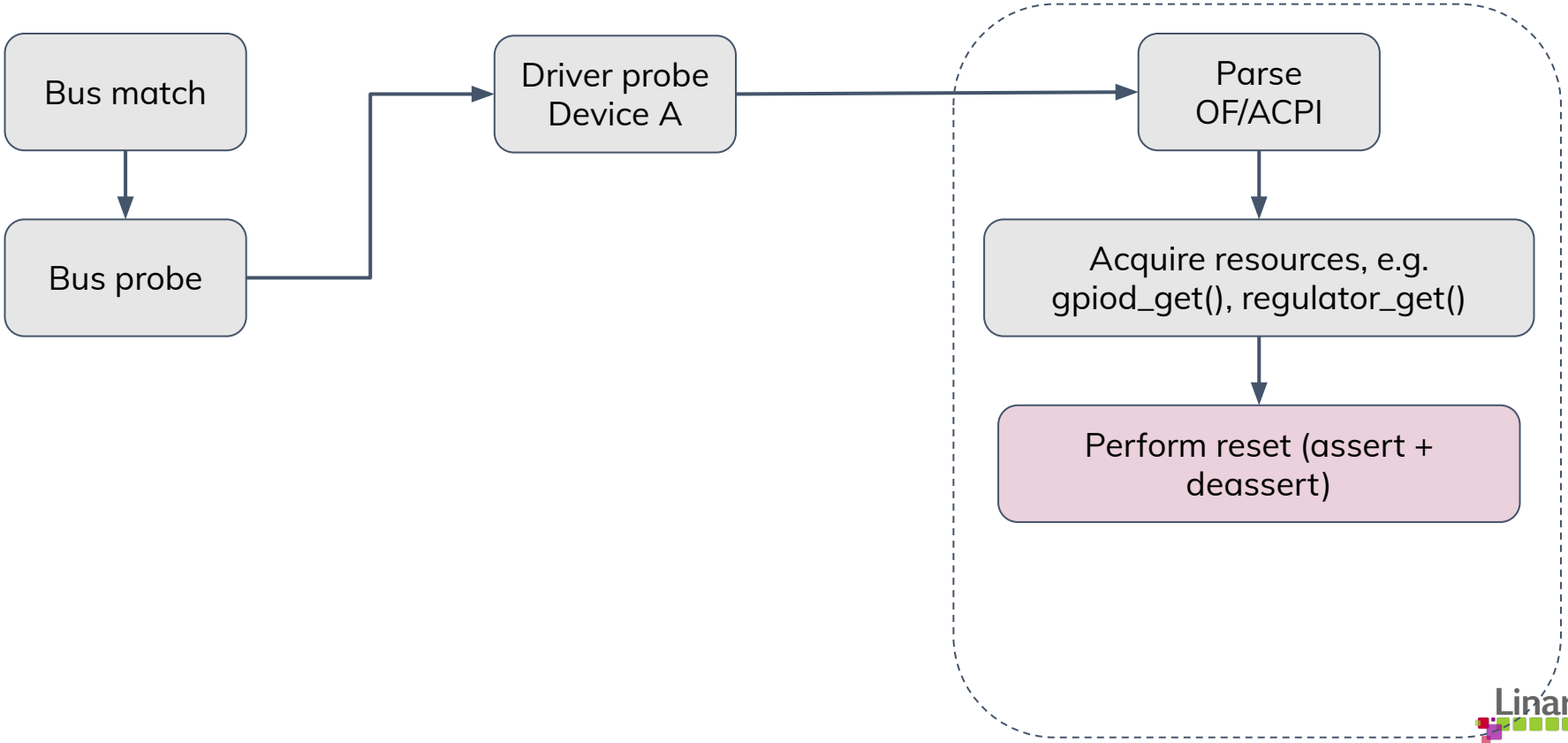
(Simplified) Linux Device Driver Probing



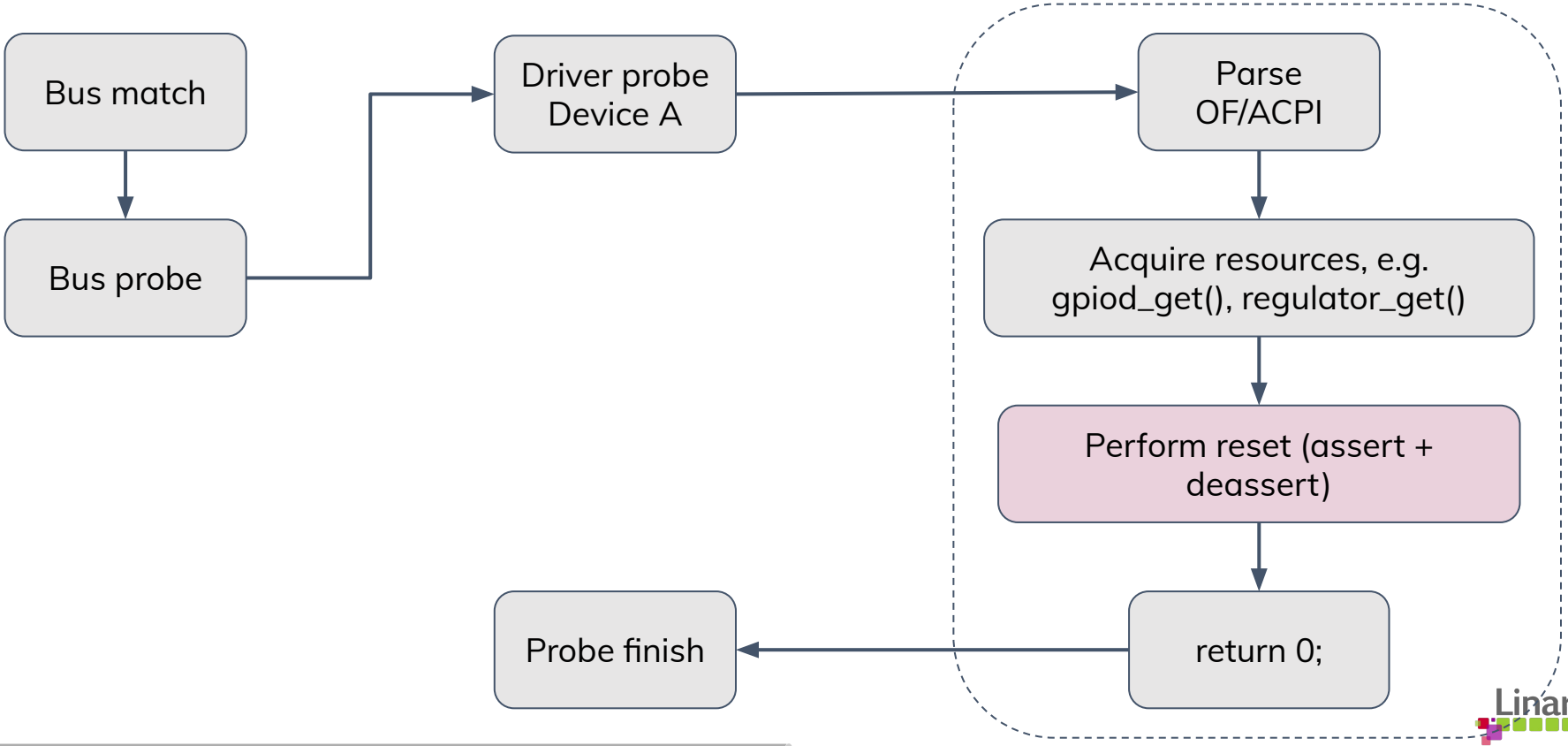
(Simplified) Linux Device Driver Probing



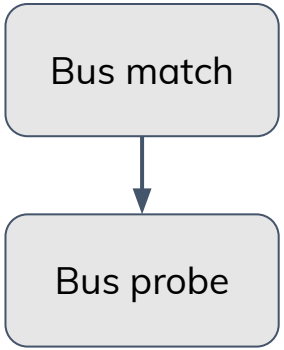
(Simplified) Linux Device Driver Probing



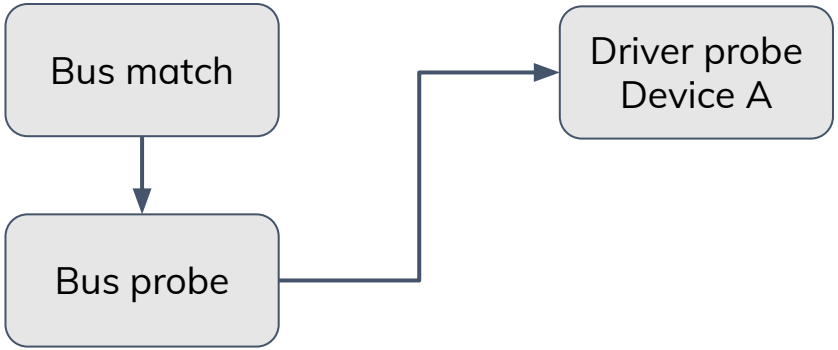
(Simplified) Linux Device Driver Probing



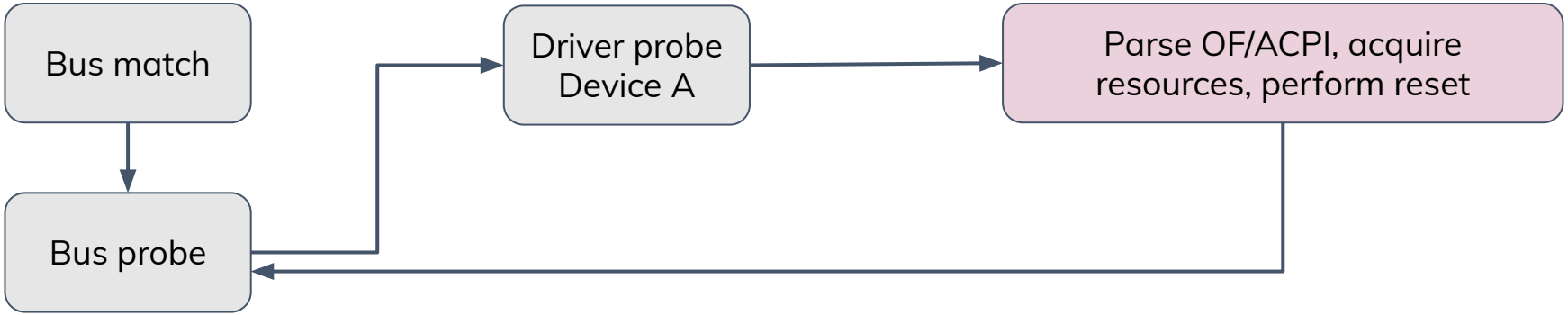
(Simplified) Linux Device Driver Probing



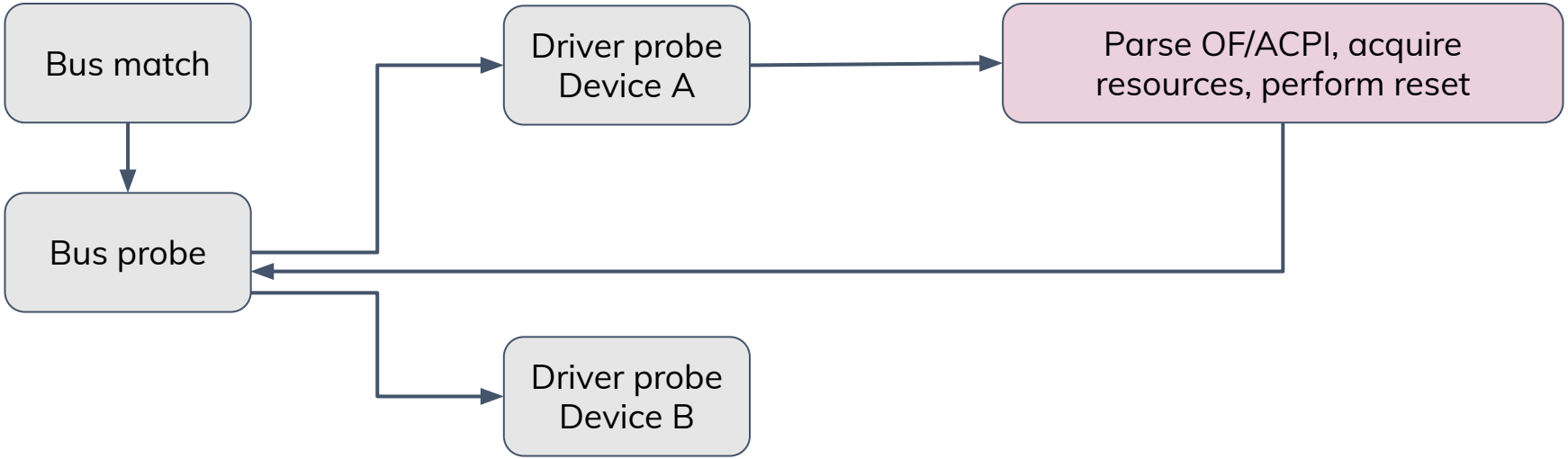
(Simplified) Linux Device Driver Probing



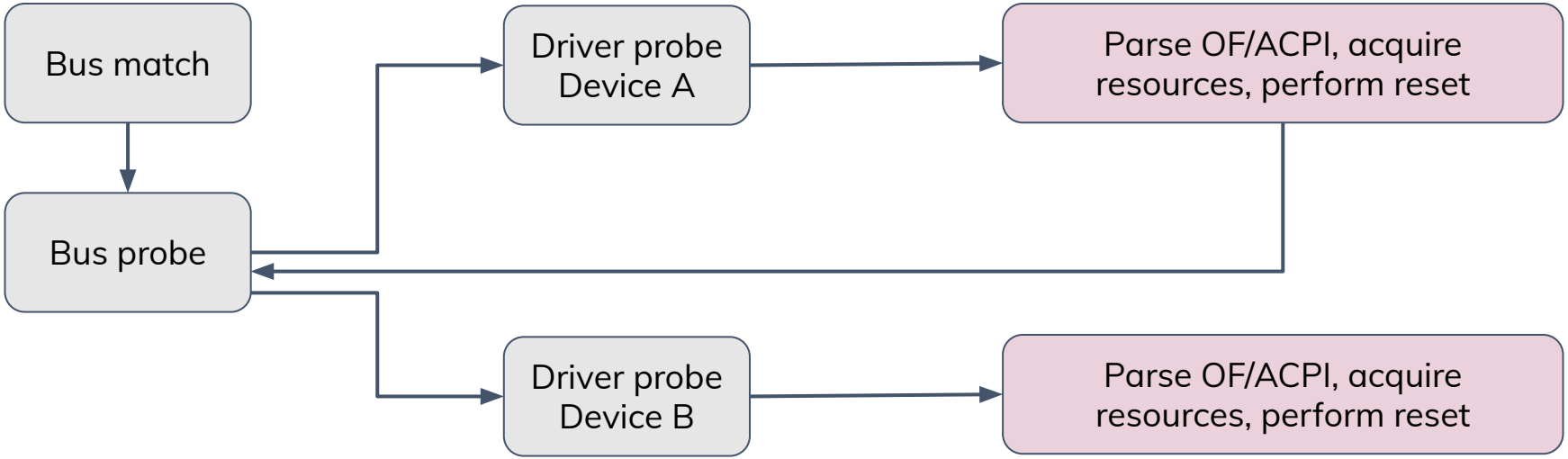
(Simplified) Linux Device Driver Probing



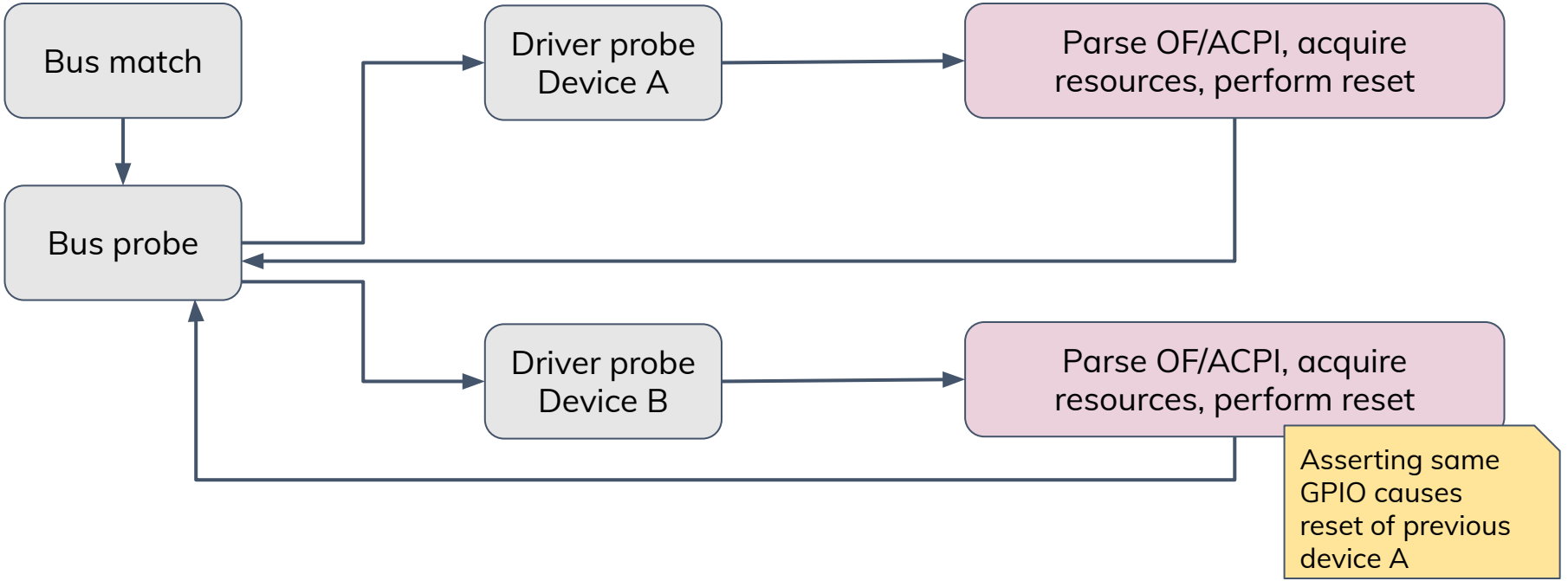
(Simplified) Linux Device Driver Probing



(Simplified) Linux Device Driver Probing



(Simplified) Linux Device Driver Probing



Solution: Non-exclusive GPIO

Requesting GPIO Twice

- GPIOs are exclusive

Requesting GPIO Twice

- GPIOs are exclusive
- Two Linux devices requesting same GPIO:

```
wsa884x-codec sdw:1:0:0217:0204:00:0: error -EBUSY: Cannot get GPIO
```

- GPIO framework by default does not handle multiple users (drivers) of the same GPIO

Requesting GPIO Twice: Non-exclusive

- But there is a solution: GPIOD_FLAGS_BIT_NONEXCLUSIVE

```
sd_n = devm_gpiod_get_optional(dev, "powerdown",
                               GPIOD_FLAGS_BIT_NONEXCLUSIVE | GPIOD_OUT_HIGH);
```

Requesting GPIO Twice: Non-exclusive

- But there is a solution: GPIOD_FLAGS_BIT_NONEXCLUSIVE

```
sd_n = devm_gpiod_get_optional(dev, "powerdown",
                               GPIOD_FLAGS_BIT_NONEXCLUSIVE | GPIOD_OUT_HIGH);
```

- Wrong!
  - Non-exclusive GPIO was added for regulator framework
    - Which internally handles non-exclusive and simultaneous access
    - Exposes an enable-counting interface to the users

Requesting GPIO Twice: Non-exclusive

- But there is a solution: GPIOD_FLAGS_BIT_NONEXCLUSIVE

```
sd_n = devm_gpiod_get_optional(dev, "powerdown",
                               GPIOD_FLAGS_BIT_NONEXCLUSIVE | GPIOD_OUT_HIGH);
```

- Wrong!
  - Non-exclusive GPIO was added for regulator framework
    - Which internally handles non-exclusive and simultaneous access
    - Exposes an enable-counting interface to the users
  - That's a solution crafted only for frameworks handling simultaneous usage of GPIOs
  - Should not be used by individual drivers

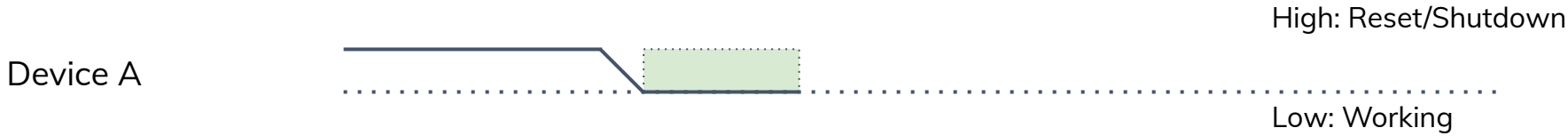
Requesting GPIO Twice: Non-exclusive

- But there is a solution: GPIOD_FLAGS_BIT_NONEXCLUSIVE

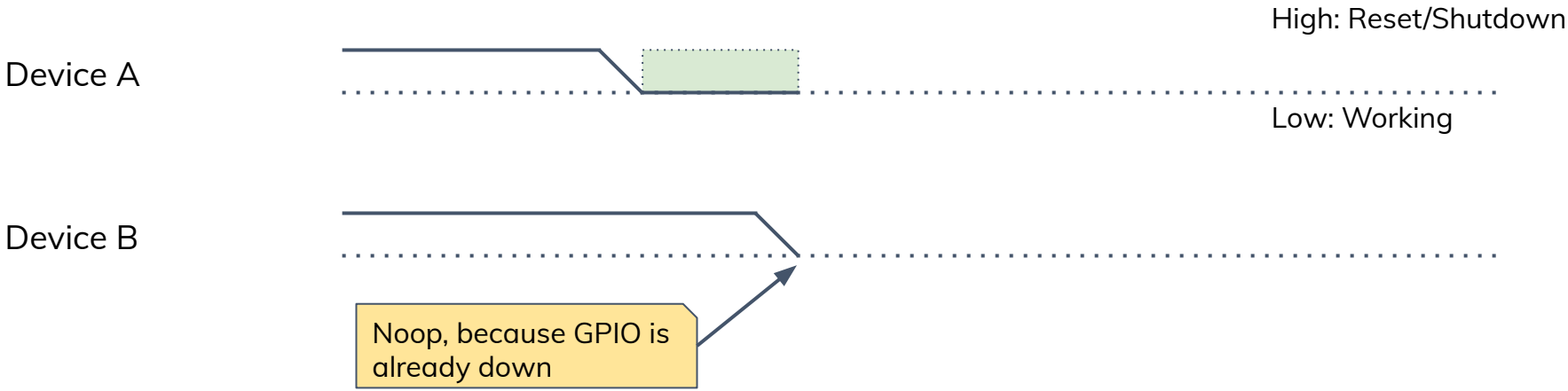
```
sd_n = devm_gpiod_get_optional(dev, "powerdown",
                               GPIOD_FLAGS_BIT_NONEXCLUSIVE | GPIOD_OUT_HIGH);
```

- Wrong!
  - Non-exclusive GPIO was added for regulator framework
    - Which internally handles non-exclusive and simultaneous access
    - Exposes an enable-counting interface to the users
  - That's a solution crafted only for frameworks handling simultaneous usage of GPIOs
  - Should not be used by individual drivers
  - Does not solve problem of one device resetting the other one

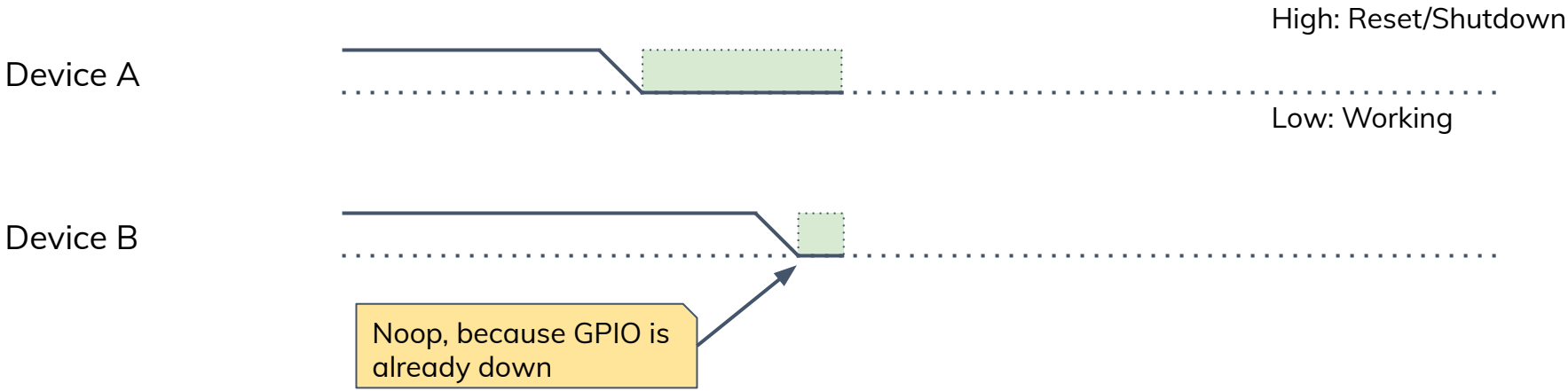
Dumb Drivers with FLAGS_BIT_NONEXCLUSIVE



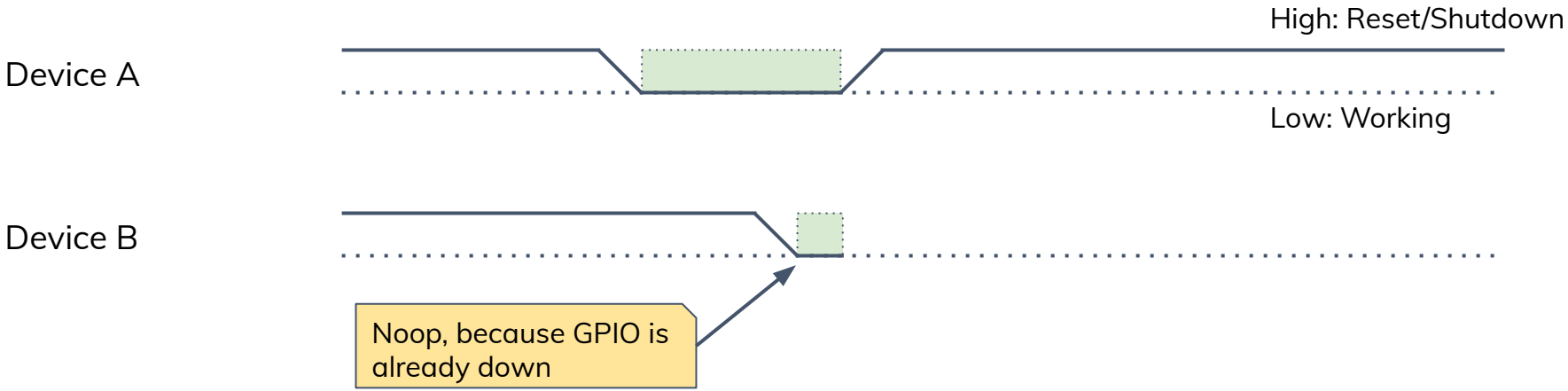
Dumb Drivers with FLAGS_BIT_NONEXCLUSIVE



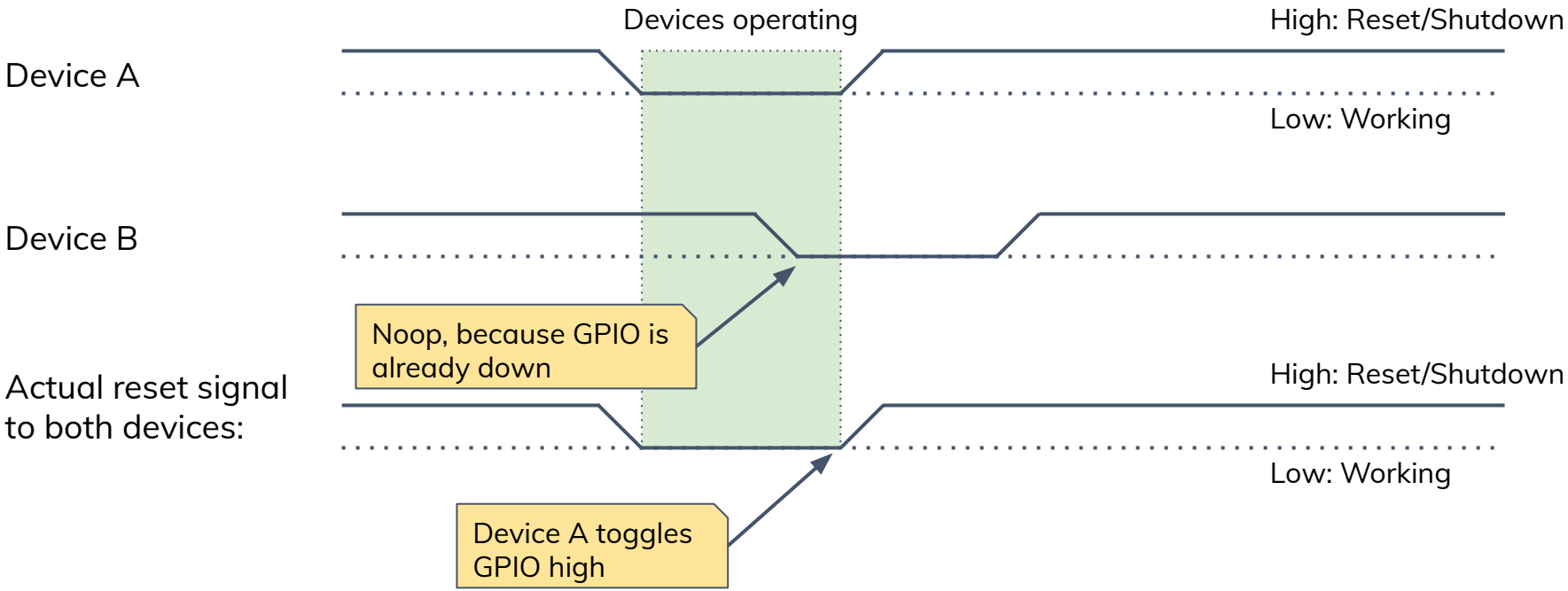
Dumb Drivers with FLAGS_BIT_NONEXCLUSIVE



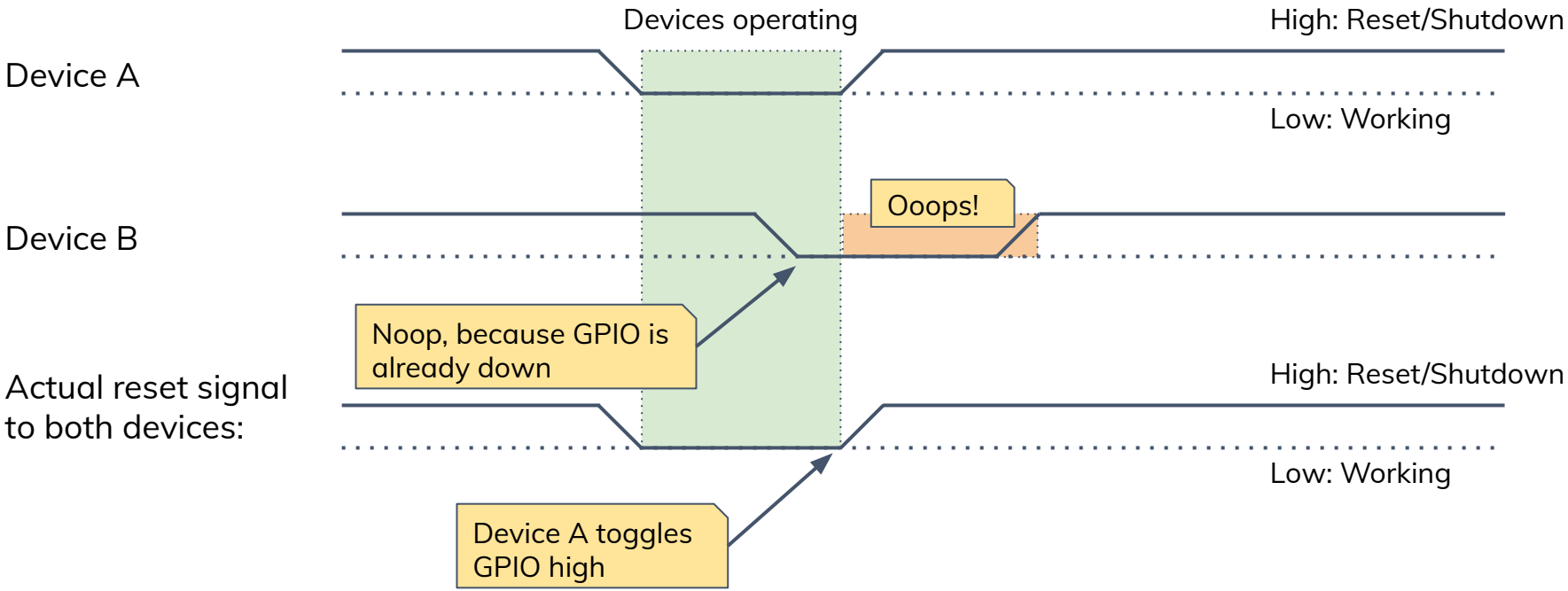
Dumb Drivers with FLAGS_BIT_NONEXCLUSIVE



Dumb Drivers with FLAGS_BIT_NONEXCLUSIVE



Dumb Drivers with FLAGS_BIT_NONEXCLUSIVE



Linaro

Regulators

- GPIOD_FLAGS_BIT_NONEXCLUSIVE was added for the Regulator framework

Regulators

- GPIOD_FLAGS_BIT_NONEXCLUSIVE was added for the Regulator framework
- Regulators handle it internally, thus using the same GPIO is perfectly fine

```
regulator-1 {
        compatible = "regulator-fixed";
        regulator-name = "foo-1";
        gpios = <&tlmm 17 GPIO_ACTIVE_HIGH>;
        enable-active-high;
};

regulator-2 {
        compatible = "regulator-fixed";
        regulator-name = "foo-2";
        gpios = <&tlmm 17 GPIO_ACTIVE_HIGH>;
        enable-active-high;
};
```

Solution: Reset Framework (v6.9-rc1)

Coming Soon with v6.9: reset-gpio

- Linux v6.9-rc1 merged my work of using Reset framework to handle reset GPIOs seamlessly
  - For shared reset/shutdown lines

Coming Soon with v6.9: reset-gpio

- Linux v6.9-rc1 merged my work of using Reset framework to handle reset GPIOs seamlessly
  - For shared reset/shutdown lines
  - Not yet solving shared reset pulses

Coming Soon with v6.9: reset-gpio

- Linux v6.9-rc1 merged my work of using Reset framework to handle reset GPIOs seamlessly
  - For shared reset/shutdown lines
  - Not yet solving shared reset pulses
- Drivers no longer request GPIO (through GPIOLIB)
  - Instead request a reset line, via reset controller framework

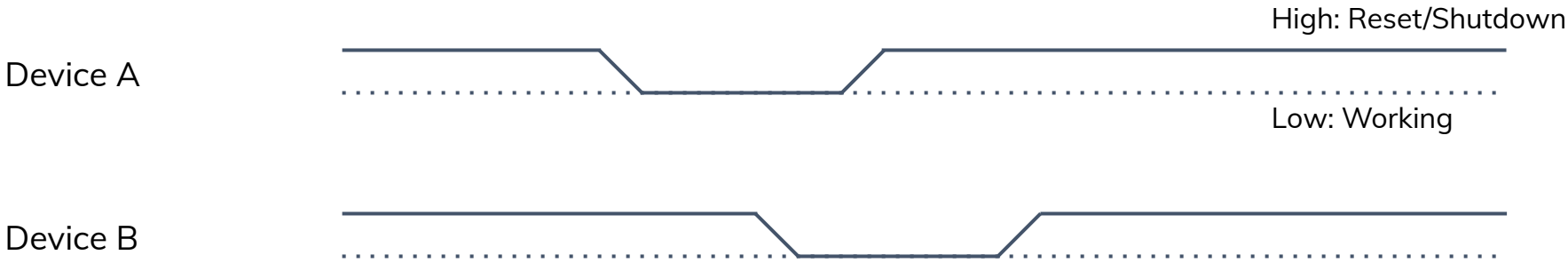
Coming Soon with v6.9: reset-gpio

- Linux v6.9-rc1 merged my work of using Reset framework to handle reset GPIOs seamlessly
  - For shared reset/shutdown lines
  - Not yet solving shared reset pulses
- Drivers no longer request GPIO (through GPIOLIB)
  - Instead request a reset line, via reset controller framework
  - If device node does not have “resets”, but has a “reset-gpio”, the Reset framework will use that GPIO through a reset-gpio driver
    - Ad-hoc instantiated reset-gpio device

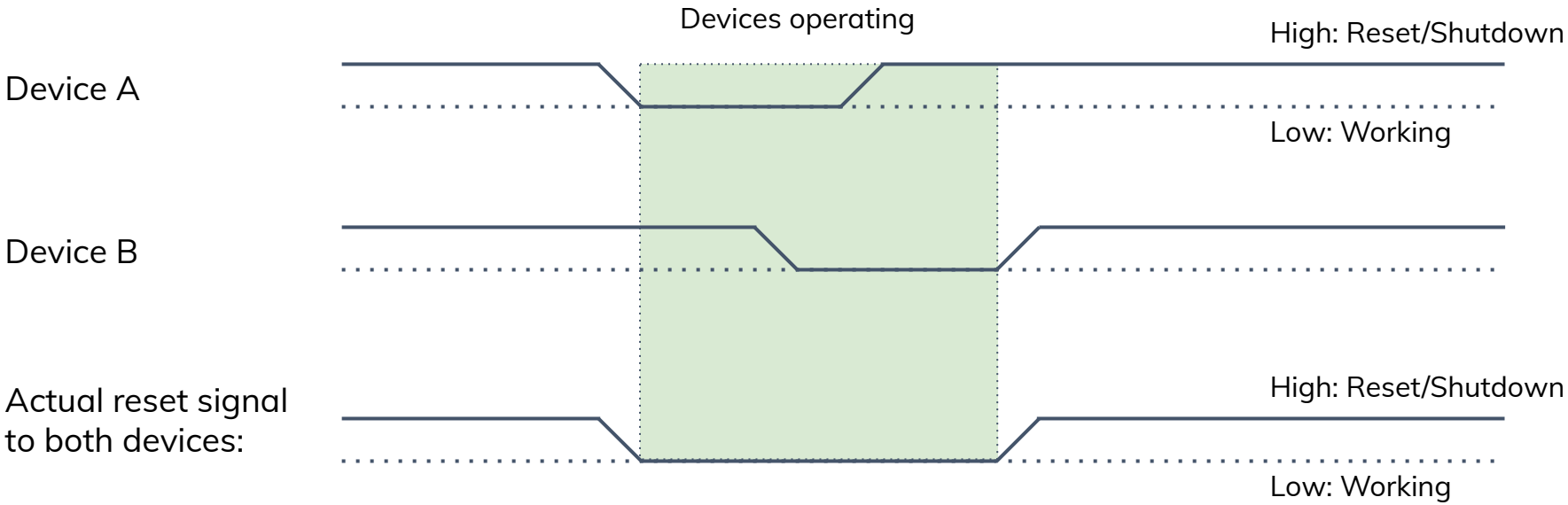
Coming Soon with v6.9: reset-gpio

- Linux v6.9-rc1 merged my work of using Reset framework to handle reset GPIOs seamlessly
  - For shared reset/shutdown lines
  - Not yet solving shared reset pulses
- Drivers no longer request GPIO (through GPIOLIB)
  - Instead request a reset line, via reset controller framework
  - If device node does not have “resets”, but has a “reset-gpio”, the Reset framework will use that GPIO through a reset-gpio driver
    - Ad-hoc instantiated reset-gpio device
- Limited only to Devicetree platforms
- Examples:
  - drivers/i2c/muxes/i2c-mux-pca954x.c
  - sound/soc/codecs/wsa884x.c

Shared Reset or Shutdown Line, Active High



Shared Reset or Shutdown Line, Active High



reset-gpio: Devicetree

```
      left_tweeter: speaker@0,1 {
            compatible = "sdw20217020400";
            reg = <0 1>;
-           powerdown-gpios = <&lpass_tlmm 12 GPIO_ACTIVE_LOW>;
+           reset-gpios = <&lpass_tlmm 12 GPIO_ACTIVE_LOW>;
            #sound-dai-cells = <0>;
            sound-name-prefix = "TwitterLeft";
            vdd-1p8-supply = <&vreg_l15b_1p8>;
```

No “resets” property

reset-gpio: Driver Probe

```
static int get_reset(struct device *dev, ...)
{
      ...









      sd_n = devm_gpiod_get_optional(dev, "powerdown", GPIOD_OUT_HIGH);
      if (IS_ERR(sd_n))
            return dev_err_probe(dev, PTR_ERR(sd_n),
                                 "Shutdown Control GPIO not found\n");

      return 0
};
```

reset-gpio: Driver Probe

```
 static int get_reset(struct device *dev, ...)
 {
       ...

+      sd_reset = devm_reset_control_get_optional_shared(dev, NULL);
+      if (IS_ERR(sd_reset))
+            return dev_err_probe(dev, PTR_ERR(sd_reset), "Failed to get reset\n");
+      else if (sd_reset)
+            return 0;
+      /*
+       * else: NULL, so use the backwards compatible way for powerdown-gpios,
+       * which does not handle sharing GPIO properly.
+       */
       sd_n = devm_gpiod_get_optional(dev, "powerdown", GPIOD_OUT_HIGH);
       if (IS_ERR(sd_n))
             return dev_err_probe(dev, PTR_ERR(sd_n),
                                  "Shutdown Control GPIO not found\n");

       return 0
 };
```

A shared reset!

reset-gpio: Driver Reset (De)Assert

```
 static void device_powerdown(void *data)
 {
       struct foo_priv *foo = data;

-      gpiod_direction_output(foo->sd_n, 1);
+      if (foo->sd_reset)
+            reset_control_assert(foo->sd_reset);
+      else
+            gpiod_direction_output(foo->sd_n, 1);
 }
```

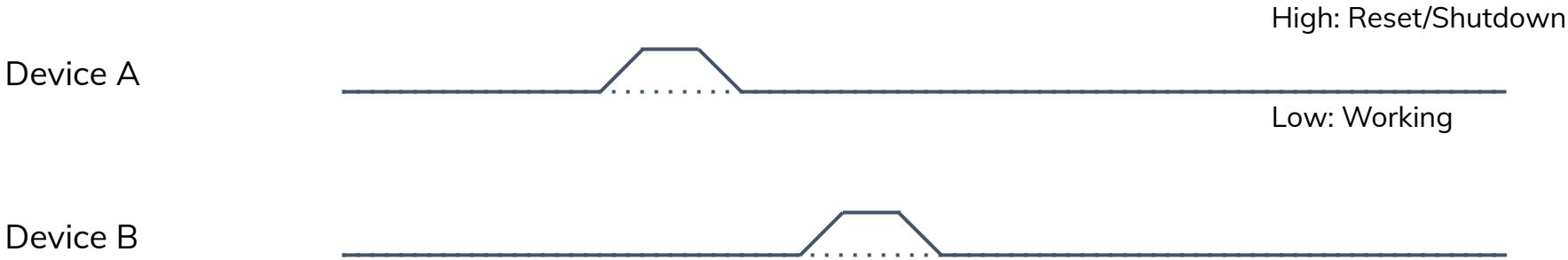
reset-gpio: Limitation on Reset Pulses

- Reset controller framework supports reset pulses
  - Slightly different API: reset_control_reset()

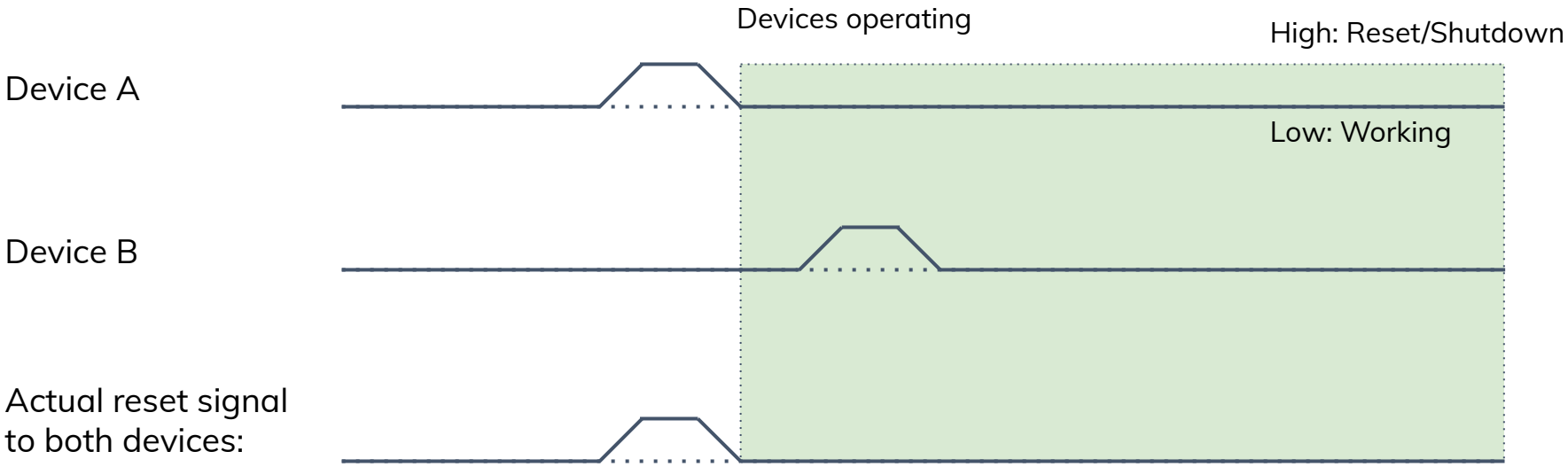
reset-gpio: Limitation on Reset Pulses

- Reset controller framework supports reset pulses
  - Slightly different API: reset_control_reset()
- Linux allows sharing such reset lines
  - To send another pulse, all reset consumers must re-arm it with reset_control_rearm()

Shared Reset Pulse, Active High



Shared Reset Pulse, Active High



reset-gpio: Limitation on Reset Pulses

- Reset controller framework supports reset pulses
  - Slightly different API: reset_control_reset()
- Linux allows sharing such reset lines
  - To send another pulse, all reset consumers must re-arm it with reset_control_rearm()
- reset-gpio driver does not implement sending pulses
  - Not implemented, on purpose

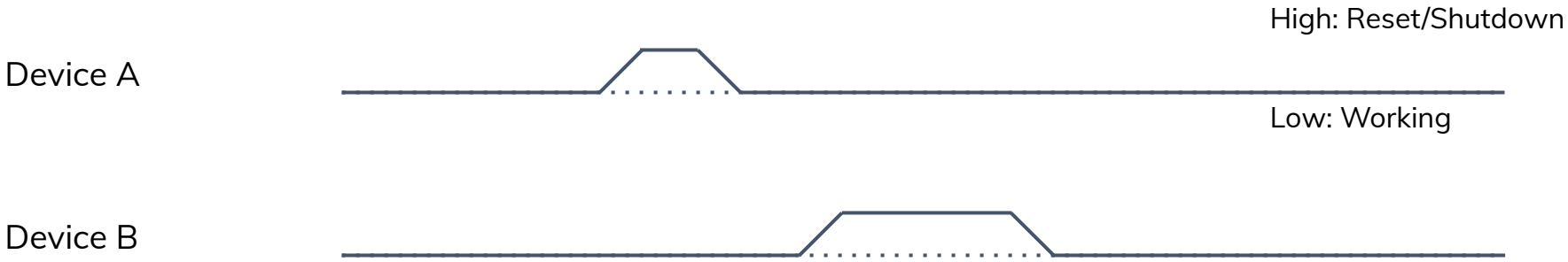
reset-gpio: Limitation on Reset Pulses

- Reset controller framework supports reset pulses
  - Slightly different API: reset_control_reset()
- Linux allows sharing such reset lines
  - To send another pulse, all reset consumers must re-arm it with reset_control_rearm()
- reset-gpio driver does not implement sending pulses
  - Not implemented, on purpose
  - Different devices might have different requirement of pulse-length

reset-gpio: Limitation on Reset Pulses

- Reset controller framework supports reset pulses
  - Slightly different API: reset_control_reset()
- Linux allows sharing such reset lines
  - To send another pulse, all reset consumers must re-arm it with reset_control_rearm()
- reset-gpio driver does not implement sending pulses
  - Not implemented, on purpose
  - Different devices might have different requirement of pulse-length
  - Support for this could be added
    - See also previous work of Sean Anderson

Shared Reset Pulse with Different Pulse Length



Shared Reset Pulse with Different Pulse Length

High: Reset/Shutdown

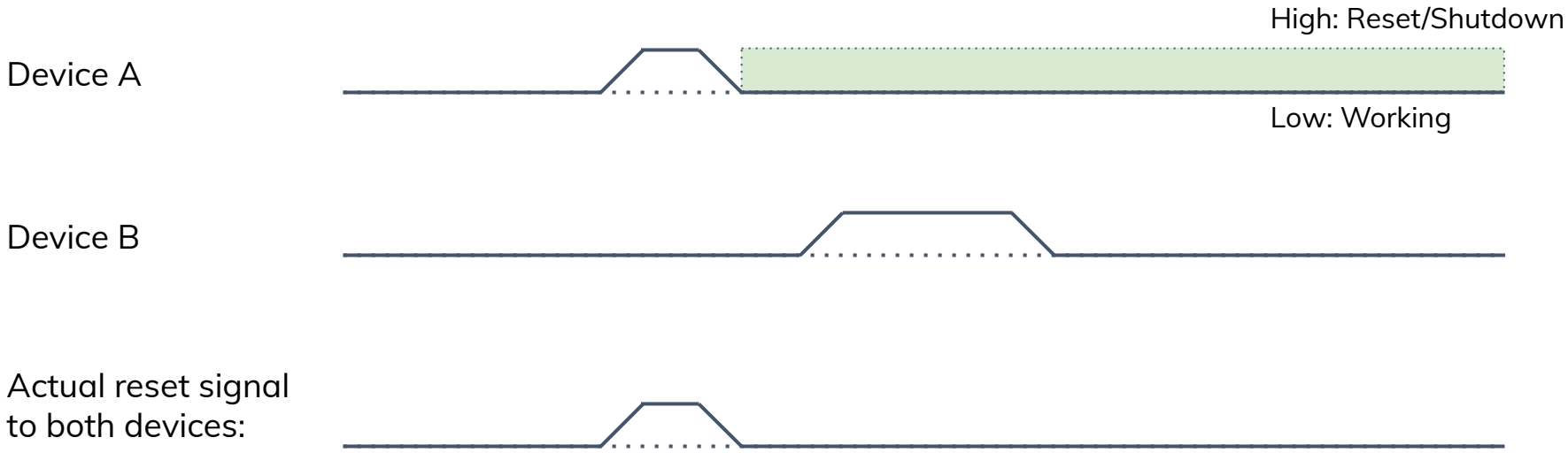
Device A

Low: Working

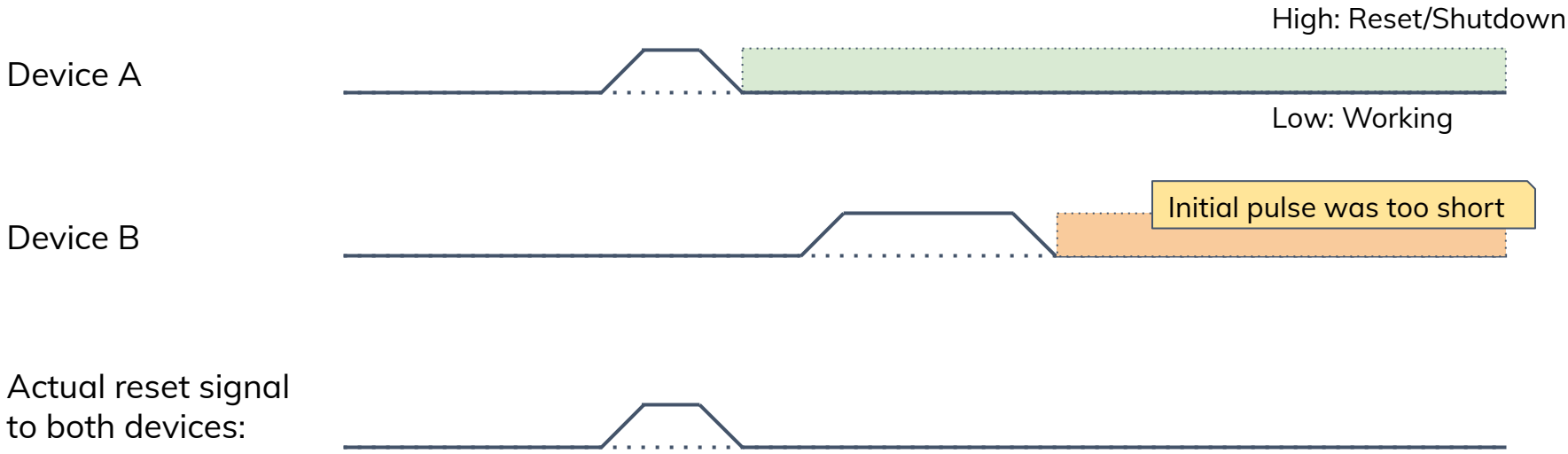
Device B

Actual reset signal
to both devices:

Shared Reset Pulse with Different Pulse Length



Shared Reset Pulse with Different Pulse Length



GPIO Aggregator and Delay

Linaro

GPIO Aggregator

- Linux driver to aggregate existing GPIOs and expose them as a new gpiochip

GPIO Aggregator

- Linux driver to aggregate existing GPIOs and expose them as a new gpiochip
- Use cases:
  - Aggregating GPIOs using /dev interface

GPIO Aggregator

- Linux driver to aggregate existing GPIOs and expose them as a new gpiochip
- Use cases:
  - Aggregating GPIOs using /dev interface
    - GPIO controllers are exported to userspace using /dev/gpiochip* character devices
    - Any access control is per entire gpiochip, so all-or-nothing

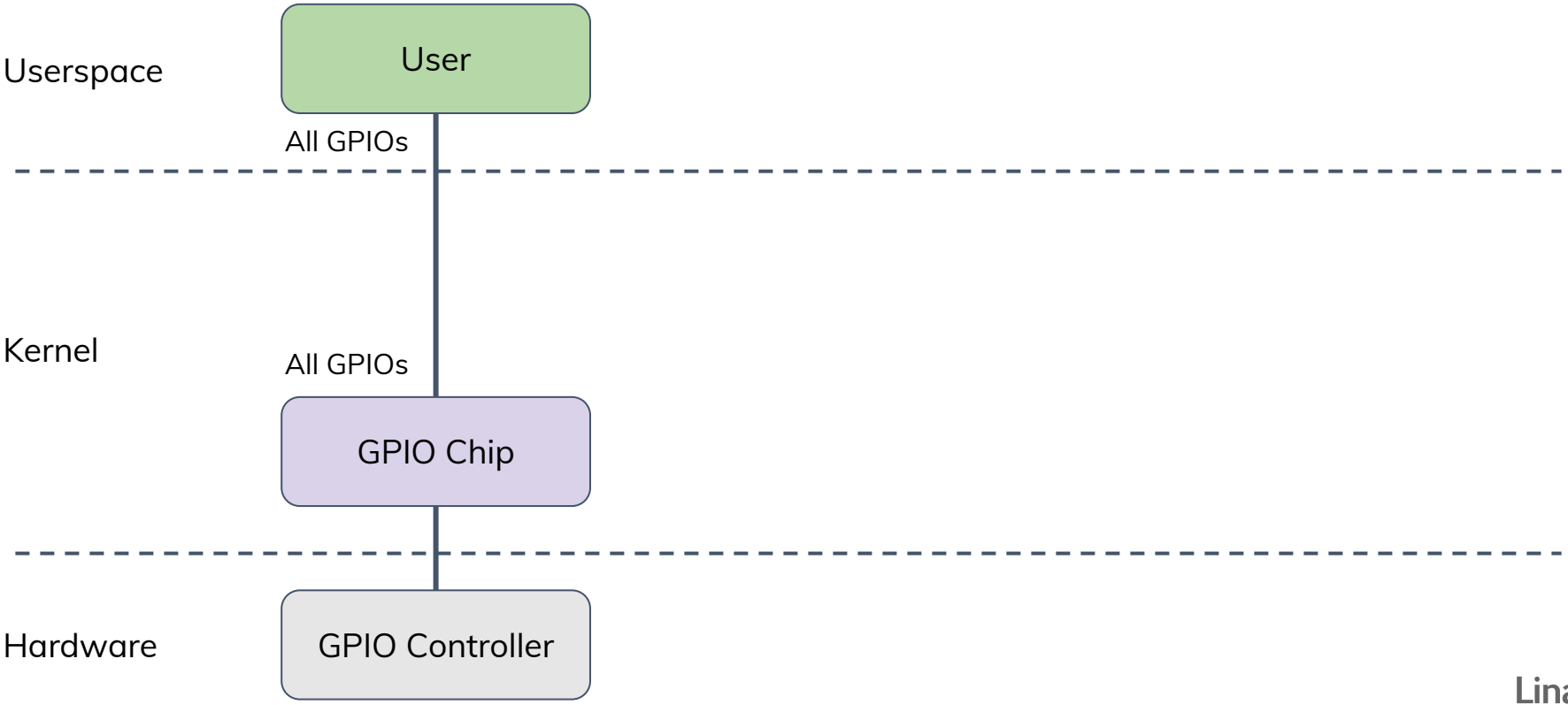
GPIO Aggregator

- Linux driver to aggregate existing GPIOs and expose them as a new gpiochip
- Use cases:
  - Aggregating GPIOs using /dev interface
    - GPIO controllers are exported to userspace using /dev/gpiochip* character devices
    - Any access control is per entire gpiochip, so all-or-nothing
    - GPIO aggregator allows to partition one GPIO controller into multiple controllers

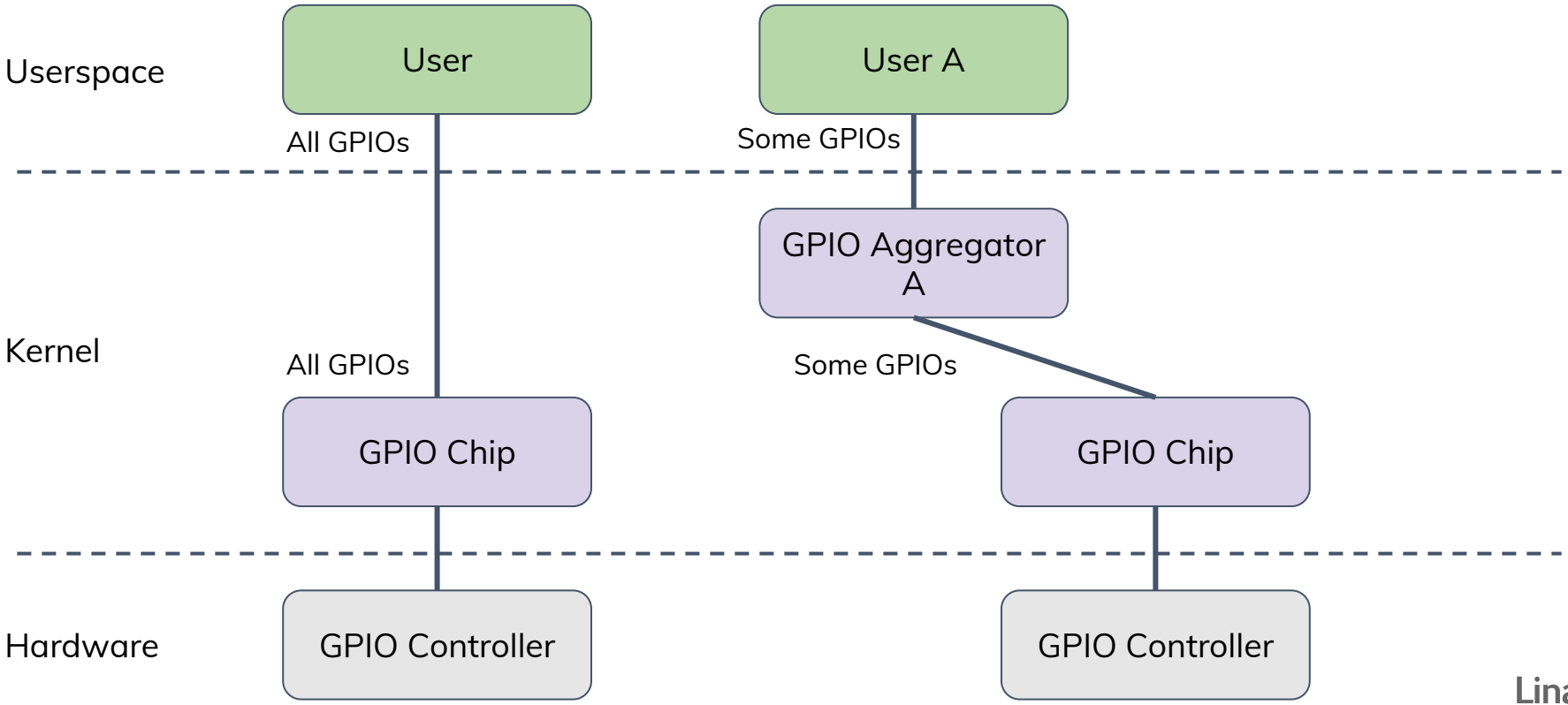
GPIO Aggregator

- Linux driver to aggregate existing GPIOs and expose them as a new gpiochip
- Use cases:
  - Aggregating GPIOs using /dev interface
    - GPIO controllers are exported to userspace using /dev/gpiochip* character devices
    - Any access control is per entire gpiochip, so all-or-nothing
    - GPIO aggregator allows to partition one GPIO controller into multiple controllers
    - Which in turn can be individually allowed per user or virtual machine

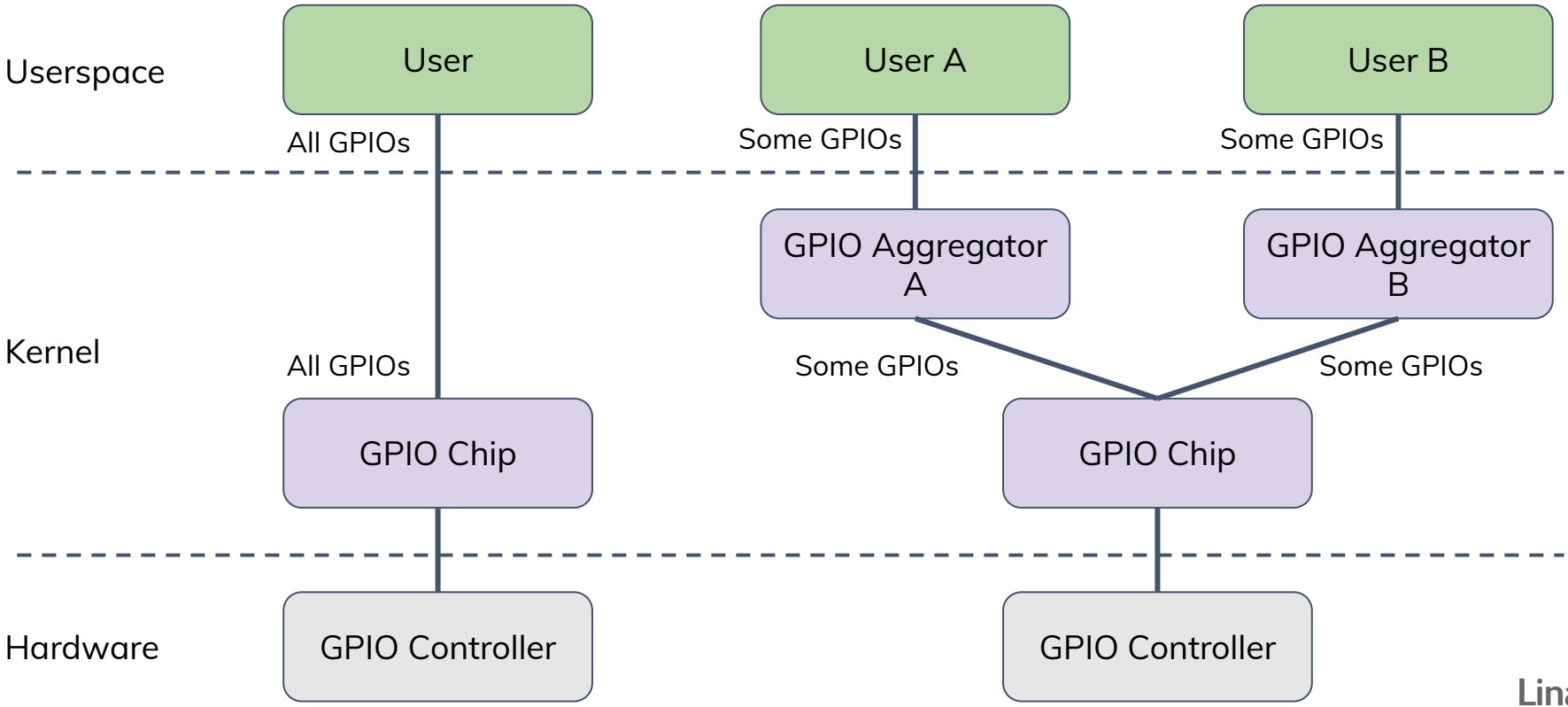
GPIO Aggregator: Access Control Example



GPIO Aggregator: Access Control Example



GPIO Aggregator: Access Control Example



GPIO Aggregator

- Linux driver to aggregate existing GPIOs and expose them as a new gpiochip
- Use cases:
  - Aggregating GPIOs using /dev interface
    - GPIO controllers are exported to userspace using /dev/gpiochip* character devices
    - Any access control is per entire gpiochip, so all-or-nothing
    - GPIO aggregator allows to partition one GPIO controller into multiple controllers
    - Which in turn can be individually allowed per user or virtual machine
  - Generic GPIO Driver
    - Industrial control, where it can provide userspace access to a simple GPIO-operated device described in Devicetree (like spidev does for SPI-operated devices)

GPIO Aggregator and Delay

- The Devicetree bindings for Aggregator are described as *gpio-delay*
  - Purpose: GPIO output delays introduced by RC circuits

GPIO Aggregator and Delay

- The Devicetree bindings for Aggregator are described as *gpio-delay*
  - Purpose: GPIO output delays introduced by RC circuits
- Currently both the driver and bindings support only 1-to-1 mapping

GPIO Aggregator and Delay

- The Devicetree bindings for Aggregator are described as *gpio-delay*
  - Purpose: GPIO output delays introduced by RC circuits
- Currently both the driver and bindings support only 1-to-1 mapping
- With a some effort driver and bindings could support 1-to-many or many-to-1 mappings
- That’s just an idea…

Unresolved Problems

Sharing GPIOs Is Not Specific to Reset

- Not only reset/shutdown GPIOs are shared

Sharing GPIOs Is Not Specific to Reset

- Not only reset/shutdown GPIOs are shared
  - TI TAS2764 audio amplifier / speakers on Apple MacBook Air have shared shutdown pin which can act as soft-shutdown

Sharing GPIOs Is Not Specific to Reset

- Not only reset/shutdown GPIOs are shared
  - TI TAS2764 audio amplifier / speakers on Apple MacBook Air have shared shutdown pin which can act as soft-shutdown
- Also enable GPIOs are clearly not “reset” GPIOs and are not handled currently

Sharing GPIOs Is Not Specific to Reset

- Not only reset/shutdown GPIOs are shared
  - TI TAS2764 audio amplifier / speakers on Apple MacBook Air have shared shutdown pin which can act as soft-shutdown
- Also enable GPIOs are clearly not “reset” GPIOs and are not handled currently
  - Devicetree: Passing enable GPIOs as “reset-gpios” property, to use new reset-gpio Linux driver, would not be accepted by Devicetree maintainer

Other Unresolved Problems

- Mentioned earlier:
  - Reset pulses with reset-gpio driver

Other Unresolved Problems

- Mentioned earlier:
  - Reset pulses with reset-gpio driver
  - Different delays for reset pulses

Other Unresolved Problems

- Mentioned earlier:
  - Reset pulses with reset-gpio driver
  - Different delays for reset pulses
- Devices needing to do something immediately after the reset

Other Unresolved Problems

- Mentioned earlier:
  - Reset pulses with reset-gpio driver
  - Different delays for reset pulses
- Devices needing to do something immediately after the reset
  - Not even sure how such driver could work, considering modules and deferred probe
  - Blame hardware engineers...

Linaro is the software engine of the Arm Ecosystem

Linaro empowers rapid product deployment within the dynamic Arm Ecosystem.

- Our cutting-edge solutions, services and collaborative platforms facilitate the swift development, testing, and delivery of Arm-based innovations, enabling businesses to stay ahead in today's competitive technology landscape.
- Our expertise and contributions spread from Testing & LTS, Security, Cloud & Edge Computing, IoT, AI, CI/CD, Toolchain and Virtualization to vertical projects like Windows on Arm and Android Ecosystem enabling and maintenance.
- Linaro fosters and environment of collaboration, standardization and optimization among businesses and open source ecosystems to accelerate the deployment of Arm-based products and technologies along with representing a pivotal role in open source discovery and adoption.

Linaro has enabled trust, quality and collaboration since 2010

Linaro

Thank you

Visit www.linaro.org
Krzysztof Kozlowski
@krzk@social.kernel.org